Distributed Systems

Mihir Rao

Contents

1 Introduction **2**
1.1 About distributed systems . . . 2
1.2 Distributed systems and computer networking . . . 3
1.3 Example: Remote Procedure Calls (RPC) . . . 4

2 Models of Distributed Systems **7**
2.1 The two generals problem . . . 7
2.2 The Byzantine generals problem . . . 8
2.3 Describing nodes and network behaviour . . . 9
2.4 Fault tolerance and high availability . . . 12

3 Time, Clocks, and Ordering of Events **14**
3.1 Physical clocks . . . 14
3.2 Clock synchronisation and monotonic clocks . . . 16
3.3 Causality and happens-before . . . 18

4 Broadcast Protocols and Logical Time **20**
4.1 Logical time . . . 20
4.2 Delivery order in broadcast protocols . . . 23
4.3 Broadcast algorithms . . . 25

5 Replication **28**
5.1 Manipulating remote state . . . 28
5.2 Quorums . . . 30
5.3 Replication using broadcast . . . 32

6 Consensus **34**
6.1 Introduction to consensus . . . 34
6.2 The Raft consensus algorithm . . . 36

7 Replica Consistency **42**
7.1 Two-phase commit . . . 42
7.2 Linearizability . . . 44
7.3 Eventual consistency . . . 47

8 Case Studies **50**
8.1 Collaboration and conflict resolution . . . 50
8.2 Google's Spanner . . . 54

Lecture 1: Introduction

A distributed system consists of multiple computers that communicate via a network while trying to achieve some task together. In Leslie Lamport's words, it is "a system in which the failure of a computer you didn't even know existed can render your own computer unusable." Each computer is a node: a desktop computer, a server in a datacenter, a phone, a car, a sensor, or any other communicating computing device.

Threads of one process share an address space, so a pointer valid in one thread is valid in another (shared-memory concurrency). Nodes of a distributed system still run concurrently, but each runs its own operating system with its own memory, so they can communicate only by sending messages over a network. Limited forms of distributed shared memory exist (some supercomputers, remote direct memory access, and databases viewed as shared memory with a different data model), but most practical distributed systems are based on message passing.

1.1 About distributed systems

There are four main reasons to make a system distributed.

- *It is inherently distributed:* sending a message from your phone to a friend's phone requires the two phones to communicate.
- *Better reliability:* a single computer fails and must sometimes be rebooted, but with several nodes one can keep serving users while another reboots.
- *Better performance:* users all over the world can be routed to a nearby node instead of one halfway round the world, getting around the speed of light.
- *Bigger problems:* some data or computations do not fit on one machine. The Large Hadron Collider at CERN is supported by a worldwide infrastructure with 1 million CPU cores and 1 exabyte (10^{18} bytes) of storage.

The downside is that things go wrong. Communication may fail, and we might not even know it has failed; a node may crash, run much slower than usual, or misbehave because of a software bug or hardware failure, and we might not know that either; and all of this may happen nondeterministically, at any moment and without warning. In a single computer a failed component (say a faulty RAM module) usually just crashes the machine, and software is not written to cope with it. In a distributed system we often want the rest of the system to keep working when some part is broken (a partial failure).

Definition: A fault is a component of a system that stops working. Fault tolerance means that

the system as a whole continues functioning despite faults.

Dealing with faults is what makes distributed computing fundamentally different from, and often harder than, programming a single computer.

1.2 Distributed systems and computer networking

Distributed systems use a simple abstraction of communication: node i sends a message m to node j. Reality is much more complex: many network operators (eduroam, home DSL, cellular data, coffee-shop wifi, submarine cables, satellites) and many physical media (electric current, radio waves, lasers, even hard drives in a van). We ignore how a message is represented on the wire, because the principle of sending and receiving messages stays the same as networking technologies come and go. To send tens of terabytes it is in fact faster to load hard drives into a van and drive them to their destination: a channel with high latency but high bandwidth.

Definition: LATENCY is the delay from the time a message is sent until it is received. BANDWIDTH is the volume of data that can be transferred per unit time.

Very rough numbers, which vary hugely in practice:

Latency		Bandwidth	
Same building or datacenter	≈ 1 ms	3G cellular data	≈ 1 Mbit/s
One continent to another	≈ 100 ms	Home broadband	≈ 10 Mbit/s
Hard drives in a van	≈ 1 day	Hard drives in a van	50 TB/box ≈ 1 Gbit/s

Computer networking studies the protocols that get messages to their destination. Distributed systems build on that facility and study how several nodes should coordinate to achieve a shared task: the design of a DISTRIBUTED ALGORITHM is about deciding what messages to send and how to process messages when they are received.

The web is a distributed system used every day. SERVERS host websites and CLIENTS (web browsers) display them. To load a page, the browser sends an HTTP REQUEST message to the server, which replies with a RESPONSE message containing the page. We draw such exchanges as space-time diagrams: each node is a vertical line, time flows from top to bottom, and a message is an arrow from the sending event to the receiving event. In a URL such as `https://www.cst.cam.ac.uk/teaching/2122/ConcDisSys`, the part between `//` and the next `/` is the HOSTNAME of the server and the rest is the PATH requested. The request also carries the HTTP method (`GET` to load a page, `POST` to submit a form), the client software version (the user-agent), and the file formats the client understands (the accept header); the response carries the file and an indicator of its format (the content-type). HTTP runs on top of TCP, which splits a large chunk of data into a stream of small packets and reassembles them at the recipient, and several requests and responses may share one TCP connection. From a distributed-systems point of view we nevertheless

treat the request as one message and the response as another, independently of the underlying networking technology.

1.3 Example: Remote Procedure Calls (RPC)

When you pay in an online shop, the shop sends a payment request over the Internet to a payments service, which in turn communicates with a card network such as Visa or MasterCard, which communicates with the bank that issued the card. The shop's code might look like this:

```
// Online shop handling customer's card details
Card card = new Card();
card.setCardNumber("1234 5678 8765 4321");
card.setExpiryDate("10/2024");
card.setCVC("123");

Result result = paymentsService.processPayment(card,
    3.99, Currency.GBP);

if (result.isSuccess()) {
    fulfilOrder();
}
```

The call to `processPayment` looks like any other function call, but its implementation is not part of the shop's code: it belongs to the payments service, another program running on another node of a different company. Behind the scenes, the shop sends a request to the payments service, waits for a response, and returns the response it received.

Definition: A Remote Procedure Call (RPC) is an interaction in which code on one node appears to call a function on another node; in Java it is called Remote Method Invocation (RMI). Software that implements RPC is an RPC framework or middleware (not all middleware is based on RPC). In place of the remote function, the framework provides a stub with the same type signature, which encodes the arguments in a message (marshalling) and sends it to the remote node, asking for the function to be called.

On the server side, the framework unmarshals (decodes) the message and calls the function with the given arguments; its return value is marshalled, sent back, unmarshalled by the client, and returned by the stub (figure 1). To the caller it looks as if the function had executed locally. If the request is sent over HTTP, the RPC is also called a web service.

Ideally RPC makes a remote call look the same as a local one; hiding where a resource is located is called location transparency. In practice many things can go wrong: the service may crash during the call, and a message may be lost or delayed. If the client sends a request and gets no response, it does not know whether the server received and processed the request. Resending it might perform the request twice (charging a credit card twice), the retries may not get through either, and waiting

forever is not an option, so in practice the client gives up after a TIMEOUT.

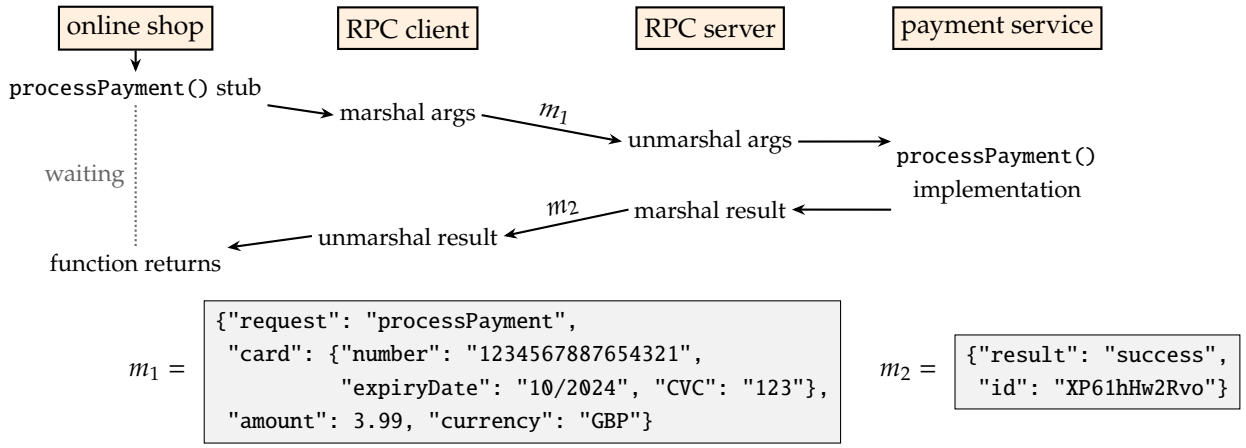


Figure 1: An RPC to `processPayment`: the stub marshals the arguments into the request m_1 (here JSON), and the result returns as m_2 while the caller waits.

Many variants of RPC have been developed over the decades to make distributed programming easier, but the underlying distributed-systems challenges have remained the same. They include SunRPC/ONC RPC (1980s, the basis for NFS); CORBA, object-oriented middleware popular in the 1990s, and the similar Microsoft DCOM and Java RMI; SOAP/XML-RPC, RPC using XML and HTTP (1998); Thrift (Facebook, 2007); gRPC (Google, 2015); REST (often with JSON); and Ajax in web browsers.

Today the most common form of RPC sends JSON over HTTP. A popular set of design principles for such APIs is REPRESENTATIONAL STATE TRANSFER (REST); APIs that follow it are RESTFUL. Communication is stateless (each request is self-contained and independent of other requests); resources (objects that can be inspected and manipulated) are represented by URLs; and the state of a resource is updated by an HTTP request with a standard method, such as `POST` or `PUT`, to its URL. REST is popular because JavaScript in a web browser can easily make such requests without reloading the page (a technique called AJAX):

```
let args = {amount: 3.99, currency: 'GBP', /*...*/ };
let request = {method: 'POST', body: JSON.stringify(args),
               headers: {'Content-Type': 'application/json'}};
fetch('https://example.com/payments', request)
  .then((response) => {
    if (response.ok) success(response.json());
    else failure(response.status); // server error
  })
  .catch((error) => {
    failure(error); // network error
  });
```

The code marshals `args` to JSON with `JSON.stringify()` and sends it in a `POST` request. There are

three outcomes: the server returns a status code indicating success (the response is unmarshalled with `response.json()`), the server returns an error status code, or no response arrives at all (most likely a network interruption). RESTful and HTTP-based RPC APIs originated on the web but are now also used by mobile apps and for server-to-server communication.

Server-to-server RPC is especially common in large enterprises. In a SERVICE-ORIENTED ARCHITECTURE (SOA), or MICROSERVICES, a large application is split into multiple services on multiple nodes that communicate via RPC; the services are developed and administered by different teams and may be written in different languages. The RPC framework then has to convert datatypes so that the caller's arguments and the function's return value are understood on both sides. A typical solution is an INTERFACE DEFINITION LANGUAGE (IDL), which gives language-independent type signatures of the functions offered over RPC; from it, developers automatically generate marshalling code and stubs for each language. gRPC's IDL, Protocol Buffers, declares messages such as `PaymentRequest` (a card, an amount, and a currency) and `PaymentStatus`, and a service `PaymentService` with `rpc ProcessPayment(PaymentRequest) returns (PaymentStatus)`.

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 1 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 2: Models of Distributed Systems

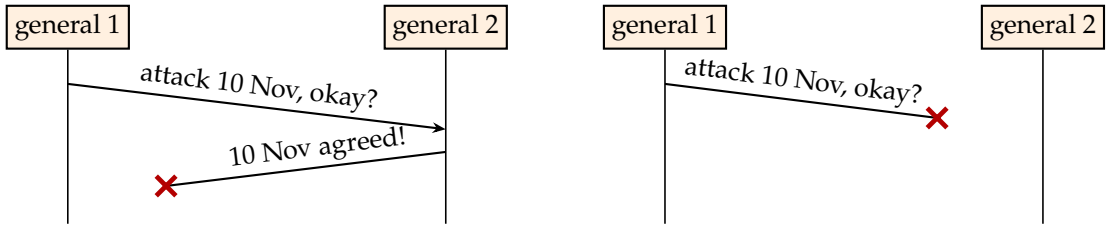
A system model captures our assumptions about how nodes and the network behave: an abstract description of their properties, which various technologies can implement in practice. Two classic thought experiments motivate the usual models.

2.1 The two generals problem

Two generals, each leading an army, want to capture a city. The city's defences are strong: an army that attacks alone is defeated, but if both attack at the same time they capture the city.

Army 1	Army 2	Outcome
does not attack	does not attack	nothing happens
attacks	does not attack	army 1 defeated
does not attack	attacks	army 2 defeated
attacks	attacks	city captured

We want army 1 to attack if and only if army 2 attacks. The armies are camped apart and communicate only by messengers who pass through territory controlled by the city and are sometimes captured. A sender does not know whether its message got through unless it receives an explicit reply, and a general who receives nothing cannot tell whether the other general sent nothing or all the messengers were captured (figure 2).



(a) the reply is lost (b) the request is lost

Figure 2: The two generals problem. From general 1's point of view, execution (a), in which general 2 agrees but the reply is captured, is indistinguishable from execution (b), in which the request never arrives.

Each general has two options. General 1 may promise to attack in any case, even with no response; sending many messengers raises the probability that one gets through, but if all are captured, general 2 does not know about the attack and general 1 attacks alone. Or general 1 may attack only after a positive response from general 2. Now general 1 is safe, but general 2 knows that general 1 attacks only if the response gets through, so general 2 is in general 1's former position: it must either

commit and risk attacking alone or wait for an acknowledgement of the acknowledgement. There is no COMMON KNOWLEDGE: the only way of knowing something is to communicate it. Repeated acknowledgements build up gradually increasing confidence, but it can be proved that no finite number of messages makes the generals certain that they agree. In a distributed system, no node can have certainty about the state of another node; it can know something only by having that knowledge communicated in a message.

As a practical instance, an online shop and a payments service communicate by RPC, and some messages may be lost. The shop wants to dispatch goods only if they are paid for, and the customer's card should be charged only if the goods are dispatched.

Online shop	Payments service	Outcome
does not dispatch	does not charge	nothing happens
dispatches	does not charge	shop loses money
does not dispatch	charges	customer complaint
dispatches	charges	everyone happy

This does not exactly match the two generals problem: a payment can be undone by a refund (unlike a defeated army), so the payments service can safely always charge. If communication is interrupted, the shop waits until the connection is restored and then queries the payments service for the status of any transaction whose outcome was unknown.

2.2 The Byzantine generals problem

Now three or more armies want to capture a city, and a sent message is always delivered correctly. However, some generals may be TRAITORS: they deliberately and maliciously try to mislead and confuse the others. We call the traitors MALICIOUS and the others HONEST. In figure 3, general 3 receives "attack" from general 1 and "general 1 said retreat" from general 2, and cannot tell whether general 2 is lying or general 1 is issuing contradictory orders.

Each general is either malicious or honest, and up to f generals may be malicious. Honest generals do not know who the malicious ones are, and the malicious generals may collude (we may even assume they are all controlled by one evil adversary). Since we cannot specify what malicious generals will do, the goal is that all honest generals agree on the same plan.

Theorem 2.1 [BYZANTINE GENERALS]. *In a system with malicious generals and unpredictable communication delays, the Byzantine generals problem can be solved only if strictly fewer than one third of the generals are malicious. That is, tolerating f malicious generals requires $3f + 1$ generals in total.*

For example, 4 generals can tolerate $f = 1$ malicious general and 7 can tolerate $f = 2$. Cryptography (digital signatures) makes the problem somewhat easier, since general 2 could then prove to general 3 what general 1 ordered, but even with signatures it remains hard.

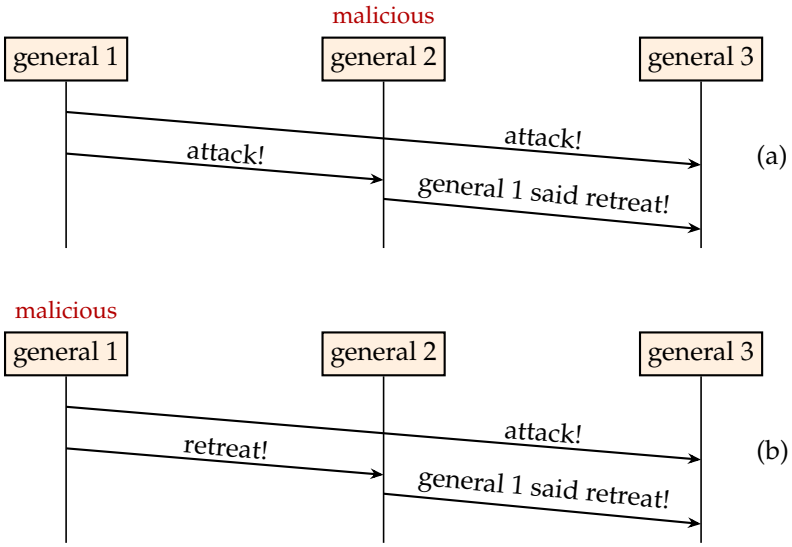


Figure 3: Generals who might lie. From general 3's point of view, execution (a), in which general 2 lies, is indistinguishable from execution (b), in which general 1 sends contradictory orders.

Real systems involve such complex trust relationships. A customer must trust an online shop to deliver the goods, though the payment can be disputed via the bank. The shop must assume customers are potentially malicious, since any way of ordering without paying would be exploited by fraudsters. Services of the same company in the same datacenter can probably trust each other; the payments service does not fully trust the shop (it may be fraudulent or use stolen card numbers), but the shop probably trusts the payments service. Yet the customer, the shop, and the payments service all need to agree on any order placed. Systems that explicitly tolerate nodes controlled by a malicious actor are BYZANTINE FAULT TOLERANT; the idea has become popular with blockchains and cryptocurrencies, which aim to provide guarantees even if some participants actively try to cheat. The word "Byzantine" had long been used in the sense of "excessively complicated, bureaucratic, devious" ("the Byzantine tax law") before Lamport adopted it; there is no evidence that the generals of the Byzantine empire were more prone to intrigue than others.

2.3 Describing nodes and network behaviour

The two generals problem models the network and the Byzantine generals problem models node behaviour; in real systems both may be faulty. A system model captures our assumptions about three parts: network behaviour (e.g. message loss), node behaviour (e.g. crashes), and timing behaviour (e.g. latency), with a choice of model for each.

Network. No network is perfectly reliable: someone unplugs the wrong cable, sharks bite undersea fibre-optic cables and cows step on cables on land, and networks get overloaded by accident or by a denial-of-service attack. Most distributed algorithms assume bidirectional message passing between a pair of nodes, called POINT-TO-POINT or UNICAST communication; some networks offer

broadcast or multicast (e.g. to discover a printer on a local network), but unicast-only is a good model of today's Internet. The link is one of the following.

- Reliable (perfect) links: a message is received if and only if it is sent. Messages may be reordered.
- Fair-loss links: messages may be lost, duplicated, or reordered, but if you keep retrying, a message eventually gets through.
- Arbitrary links (active adversary): a malicious adversary may interfere with messages (eavesdrop, modify, drop, spoof, replay).

A network partition is some links dropping or delaying all messages for an extended period of time.

Some link types can be converted into others. A fair-loss link becomes a reliable link by retransmitting lost messages until they are received and filtering out duplicates at the recipient: the fair-loss assumption means a partition lasts only for a finite time, so every message is eventually received (perhaps long after the partition is repaired; these definitions say nothing about latency). TCP performs this retry and deduplication at the packet level, but usually gives up after a timeout on the order of one minute, so surviving longer partitions needs a separate retry and deduplication mechanism on top of TCP. An arbitrary link models the Internet accurately, since the operator of any network you pass through can manipulate your packets. Cryptography (TLS, the "s" in `https://`) prevents an active adversary from eavesdropping, modifying, spoofing, or replaying traffic, but not from blocking it, so an arbitrary link becomes a fair-loss link only if the adversary does not block communication forever. Thus reliable links are less unrealistic than they seem, provided we are willing to wait arbitrarily long for retries during a partition. However, a sender may crash while retransmitting, and the message may then be permanently lost.

Nodes. Each node executes a specified algorithm, under one of the following assumptions.

- Crash-stop (fail-stop): a node is faulty if it crashes (at any moment); after crashing it stops executing forever.
- Crash-recovery (fail-recovery): a node may crash at any moment, losing its in-memory state, and may resume executing sometime later. Data stored on disk survives the crash.
- Byzantine (fail-arbitrary): a node is faulty if it deviates from the algorithm. Faulty nodes may do anything, including crashing or behaving maliciously.

A node that is not faulty is called correct. Crash-stop suits an unrecoverable hardware fault (a phone dropped in the toilet); after a software crash a node could restart, but a crash-stop algorithm would have to re-admit it as a new node, in exchange for a simpler algorithm. Crash-recovery makes no assumption about how long recovery takes, and a crashed node may never recover. A bug could be classed as a Byzantine fault, but if all nodes run the same software they share the bug, and

an algorithm that assumes fewer than a third of the nodes are Byzantine cannot tolerate it (several independent implementations are rarely practical). We therefore usually reserve "Byzantine" for deliberate deviation from the protocol. Unlike link models, node models cannot be converted into one another by generic protocols: a crash-recovery algorithm may look very different from a Byzantine one.

Timing. The synchrony assumption applies to both the network and the nodes.

- Synchronous: message latency is no greater than a known upper bound, and nodes execute the algorithm at a known speed.
- Partially synchronous: the system is asynchronous for some finite (but unknown) periods of time, and synchronous otherwise.
- Asynchronous: messages can be delayed arbitrarily, nodes can pause execution arbitrarily, and there are no timing guarantees at all.

Other contexts use these words differently: for RPC and I/O, "synchronous" often means that the caller blocks until the operation completes, and "asynchronous" that it continues without waiting.

Many problems are much easier in a synchronous system, and the assumption is tempting because networks and nodes are well behaved most of the time. But most of the time is not always, and synchronous algorithms often fail catastrophically if bounded latency or bounded execution speed is violated, even briefly and rarely. Asynchronous algorithms are very robust, since temporary interruptions or latency spikes do not affect them, but some problems are impossible to solve asynchronously. The partially synchronous model is a compromise that suits many practical systems, though using it correctly requires care.

Synchrony is violated in practice in many ways. Network latency is usually predictable but occasionally increases: message loss requires retries, congestion causes queueing in switch buffers, and network or route reconfiguration can delay packets (even within one datacenter, by more than a minute). Nodes usually execute at a predictable speed, with occasional pauses: operating-system scheduling issues such as priority inversion, preemption under heavy load, stop-the-world garbage-collection pauses (up to several minutes on large heaps), and page faults, swapping, and thrashing. Real-time operating systems provide scheduling guarantees, but most distributed systems do not use them. A paused node does not notice that time "stood still", unless it checks the clock, while the other nodes carry on and may decide that it has crashed. Such a pause differs from a crash and restart, which the program handles explicitly because it loses its in-memory state and reloads its persistent state from disk. It is therefore very rarely safe to assume a synchronous model, and most distributed algorithms are designed for the asynchronous or partially synchronous model.

Network	Nodes	Timing
reliable	crash-stop	synchronous
fair-loss	crash-recovery	partially synchronous
arbitrary	Byzantine	asynchronous

Choosing one option for each part gives the system model that is the basis for any distributed algorithm. If the assumptions are wrong, all bets are off.

2.4 Fault tolerance and high availability

Definition: The AVAILABILITY (uptime) of a service is the fraction of time that it is functioning correctly. A SERVICE-LEVEL OBJECTIVE (SLO) specifies the percentage of requests that must return a correct response within a specified timeout, as measured by a certain client over a certain period, e.g. "99.9% of requests in a day get a response in 200 ms". A SERVICE-LEVEL AGREEMENT (SLA) is a contract that specifies some SLO and the consequences (e.g. refunds) if it is not met.

An online shop wants to sell at any time of day or night, so downtime means lost money and possibly breached contracts or reputation. Whether a slow response counts as "available" is somewhat arbitrary (5 seconds? 30?), which is why SLOs include a timeout. Availability is often quoted in "nines":

Nines	Uptime	Downtime
two	99%	3.7 days per year
three	99.9%	8.8 hours per year
four	99.99%	53 minutes per year
five	99.999%	5.3 minutes per year

Definition: A FAILURE means that the system as a whole is not working; a fault means that some part of it is not working. A NODE FAULT is a crash (crash-stop or crash-recovery) or a deviation from the algorithm (Byzantine); a NETWORK FAULT is dropping or significantly delaying messages. Fault tolerance means the system as a whole continues working despite faults, up to some maximum number of faults. A SINGLE POINT OF FAILURE (SPOF) is a node or network link whose fault leads to failure.

Faults are a common cause of unavailability. We can reduce their frequency (higher-quality hardware, redundancy), which never brings it to zero, or tolerate them, which is the approach of many distributed systems. Fault tolerance is always relative to a maximum number of faults: some algorithms make progress while fewer than half of the nodes have crashed, and no algorithm can get work done if all nodes crash and never recover. Fault-tolerant systems try to avoid any SPOF; the Internet has none, although losing key intercontinental fibre links causes noticeable disruption.

Tolerating faults starts with detecting them.

Definition: A FAILURE DETECTOR is an algorithm that detects whether another node is faulty. A PERFECT FAILURE DETECTOR labels a node as faulty if and only if it has crashed.

A typical failure detector for crash-stop and crash-recovery nodes periodically sends a message, awaits a response, and labels the node as crashed if no reply arrives within some timeout. By the two generals problem this is not totally accurate: it cannot tell a crashed node from a temporarily unresponsive node, a lost message, or a delayed message. (Failure detectors usually detect crash faults; Byzantine faults are not always detectable, although sometimes they leave evidence that can be used to exclude malicious nodes.) A perfect timeout-based failure detector exists only in a synchronous crash-stop system with reliable links. In a partially synchronous system no perfect failure detector exists, and in an asynchronous system no timeout-based failure detector exists at all, since timeouts are meaningless. Partially synchronous systems do, however, admit the following.

Definition: An EVENTUALLY PERFECT FAILURE DETECTOR may temporarily label a node as crashed even though it is correct, and may temporarily label a node as correct even though it has crashed, but eventually it labels a node as crashed if and only if it has crashed.

This reflects that detection is not instantaneous and that timeouts may be spurious. With such a detector, systems can recover from crashes automatically and be highly available. Tolerating crashes also eases operations: if a service tolerates one of three nodes being unavailable, a software upgrade can be rolled out by restarting one node at a time without clients noticing. Higher availability is not always better, though. It requires focussed engineering effort and conservative design: the fixed-line telephone network is designed for five nines and has been very slow to evolve. Most Internet services do not reach four nines, because beyond some point the cost of higher availability exceeds the cost of occasional downtime.

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 2 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 3: Time, Clocks, and Ordering of Events

In the night from 30 June to 1 July 2012 (UK time), many online services and systems around the world crashed simultaneously: servers locked up and stopped responding, and some airlines could not process reservations or check-ins for several hours. The cause appears in section 3.1.

Distributed systems often need to measure time: for schedulers, timeouts, failure detectors, and retry timers; for performance measurements, statistics, and profiling; to record in log files and databases when an event occurred (which helps reconstruct what happened around the same time on different nodes); for data with time-limited validity, such as cache entries; and to determine the order of events across several nodes. We distinguish two types of clock.

- Physical clocks count the number of seconds elapsed.
- Logical clocks count events, such as messages sent.

A clock in digital electronics (an oscillator) is not the same as a clock in distributed systems, which is a source of timestamps.

3.1 Physical clocks

Physical clocks (also called wall clocks) include mechanical clocks and digital clocks such as quartz clocks, found in wristwatches, computers, and phones. A quartz crystal is laser-trimmed to resonate mechanically at a specific frequency; through the piezoelectric effect (mechanical force ⇔ electric field), an oscillator circuit produces a signal at that frequency, and counting its cycles measures elapsed time. Quartz clocks are cheap but not totally accurate: manufacturing imperfections make some clocks run slightly faster than others, and the frequency varies with temperature (clocks are tuned to be stable around room temperature, and significantly higher or lower temperatures slow them down).

Definition: The drift of a clock is the rate by which it runs fast or slow, measured in parts per million (ppm): $1\,\text{ppm} = 1\,\mu\text{s/s} = 86\,\text{ms/day} = 32\,\text{s/year}$. Most computer clocks are correct to within about 50 ppm.

Atomic clocks are based on quantum-mechanical properties of atoms such as caesium or rubidium. Caesium-133 has a resonance (a "hyperfine transition") at about 9 GHz; an electronic oscillator is tuned to that frequency, and the SI second is defined as exactly 9,192,631,770 periods of this signal. Such a clock is accurate to about 1 part in 10^{14} (1 second in 3 million years) and costs around £20,000 (rubidium clocks cost about £1,000). Satellite positioning systems (GPS, Galileo, GLONASS) also provide time: 31 GPS satellites each carry an atomic clock and broadcast the current time and their location, and a receiver computes its position from the speed-of-light delays (with corrections for

atmospheric effects, relativity, and so on). A GPS receiver attached to a computer gives a clock accurate to within a fraction of a microsecond, provided it gets a clear signal; datacenters have too much electromagnetic interference, so the receiver needs an antenna on the roof.

Atomic time is disconnected from astronomical time. International Atomic Time (TAI) defines a day as $24 \times 60 \times 60 \times 9{,}192{,}631{,}770$ periods of caesium-133, but the Earth's rotation does not take exactly that long and is not even constant (it fluctuates with tides, earthquakes, glacier melting, and unexplained factors). Greenwich Mean Time originally meant solar time on the Greenwich meridian (noon is when the sun is due south); today the term UT1 refers to mean solar time at 0° longitude. The compromise is Coordinated Universal Time (UTC): TAI with corrections to account for the Earth's rotation. Time zones and daylight saving time are offsets to UTC; in the UK, GMT is defined to equal UTC and British Summer Time is UTC +1 hour.

UTC differs from TAI by leap seconds, added as needed to keep it roughly in sync with the Earth's rotation. Every year, on 30 June and 31 December at 23:59:59 UTC, one of three things happens, announced several months beforehand:

- the clock jumps immediately to 00:00:00, skipping a second (a negative leap second);
- the clock moves to 00:00:00 after one second, as usual;
- the clock moves to 23:59:60 after one second and to 00:00:00 after one further second (a positive leap second).

So in UTC a day has 86,399, 86,400, or 86,401 seconds, and an hour does not always have 3600 seconds.

A timestamp represents a particular point in time. Unix time counts seconds since 1 January 1970 00:00:00 UTC (the epoch), not counting leap seconds; Java's `System.currentTimeMillis()` is similar but counts milliseconds. ISO 8601 gives the year, month, day, hour, minute, second, and timezone offset relative to UTC, e.g. `2021-11-09T09:50:17+00:00`. Converting between the two requires the Gregorian calendar (365 days in a year, except leap years: `year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)`) and knowledge of past and future leap seconds. Computing the seconds elapsed between two timestamps requires knowing the leap seconds in between, which is impossible more than about six months ahead, because the Earth's rotation has not happened yet.

Most software, including Unix timestamps and the POSIX standard, simply ignores leap seconds. That is fine for coarse timings, but operating systems and distributed systems often need sub-second accuracy. If a Java program calls `System.currentTimeMillis()` twice, 500 ms apart, during a positive leap second (while the clock says 23:59:60), the difference cannot be 500; whether it is zero or even negative is undocumented. Poor handling of the leap second on 30 June 2012 caused the simultaneous failures above: a Linux kernel bug made the leap second likely to trigger a livelock in multithreaded processes, which a reboot did not fix but setting the system clock did. A widely used pragmatic solution is to smear the leap second: instead of inserting it between 23:59:59 and 00:00:00, clocks are deliberately slowed down over several hours before and after (or sped up for a negative

leap second). Smearing has its own problems, but it is an alternative to making all software robust to leap seconds, which may well be infeasible.

3.2 Clock synchronisation and monotonic clocks

Computers track physical time with a quartz clock (battery-powered, so it keeps running when the power is off). Because of drift, the clock error gradually increases.

Definition: The CLOCK SKEW is the difference between two clocks at a point in time. CLOCK SYNCHRONISATION periodically gets the current time from a server with a more accurate time source (an atomic clock or GPS receiver), using a protocol such as the NETWORK TIME PROTOCOL (NTP) or the PRECISION TIME PROTOCOL (PTP).

All mainstream operating systems have NTP clients built in, and many vendors run NTP servers and configure their systems to use them by default. NTP arranges clock servers into STRATA: stratum 0 is an atomic clock or GPS receiver, stratum 1 servers sync directly with a stratum 0 device, stratum 2 servers sync with stratum 1, and so on. Network latency and processing speed vary unpredictably, so a client may contact multiple servers, discard outliers, and average the rest, and makes multiple requests to the same server, using statistics to reduce the random error. This reduces clock skew to a few milliseconds in good network conditions, but it can be much worse.

To estimate the skew against one server (figure 4), the client sends a request containing its current time t_1. The server records the time t_2 when it receives the request and t_3 when it sends the response, which echoes t_1 and includes t_2 and t_3. The client records t_4 when the response arrives.

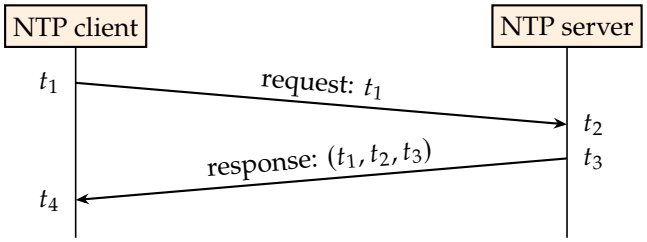


Figure 4: An NTP exchange. Times t_1 and t_4 are read from the client's clock, t_2 and t_3 from the server's clock.

The round-trip network delay is the client's round-trip time minus the server's processing time, and the client estimates the one-way latency as half of it:

$$\delta = (t_4 - t_1) - (t_3 - t_2).$$

When the response arrives, the server's clock should therefore read about $t_3 + \delta/2$, and the estimated clock skew is

$$\theta = t_3 + \frac{\delta}{2} - t_4 = \frac{t_2 - t_1 + t_3 - t_4}{2}.$$

The estimate assumes the latency is about the same in both directions. That probably holds if

latency is dominated by geographic distance, but not if queueing is significant (say one node's link is heavily loaded and the other's is not), and networks do not tell nodes the latency a particular packet actually experienced.

Having estimated the skew θ, the client corrects its clock depending on its size:

- if $|\theta| < 125\,\text{ms}$, it SLEWS the clock, speeding it up or slowing it down by up to 500 ppm, which brings the clocks in sync within about 5 minutes;
- if $125\,\text{ms} \le |\theta| < 1{,}000\,\text{s}$, it STEPS the clock, suddenly resetting it to the estimated server time, so applications watching the clock see time jump forwards or backwards;
- if $|\theta| \ge 1{,}000\,\text{s}$, it panics and does nothing, leaving the problem to a human operator.

With slewing, the client's clock frequency converges to the server's rate and the two stay in sync to within a few milliseconds, depending on the network between them. Because a client can get stuck in the panic state, running NTP does not guarantee a correct clock: systems that rely on clock synchronisation must monitor clock skew.

Stepping has an important consequence for measuring elapsed time. Java has two core functions for reading the local clock:

```
// BAD:
long startTime = System.currentTimeMillis();
doSomething();
long endTime = System.currentTimeMillis();
long elapsedMillis = endTime - startTime;
// elapsedMillis may be negative!

// GOOD:
long startTime = System.nanoTime();
doSomething();
long endTime = System.nanoTime();
long elapsedNanos = endTime - startTime;
// elapsedNanos is always >= 0
```

If NTP steps the clock during `doSomething()`, the first difference may be much larger than the true elapsed time or even negative.

Definition: A TIME-OF-DAY CLOCK (real-time clock) returns the time since a fixed date, such as the 1970 epoch. It may suddenly move forwards or backwards (NTP stepping) and is subject to leap-second adjustments; its timestamps can be compared across nodes, if they are synced. Examples: `System.currentTimeMillis()` in Java and `clock_gettime(CLOCK_REALTIME)` on Linux. A MONOTONIC CLOCK returns the time since an arbitrary point (e.g. when the machine booted). It always moves forwards at a near-constant rate (NTP slewing may adjust only the rate), so it is good for measuring elapsed time on a single node, but its timestamps are meaningless by

themselves and cannot be compared across nodes. Examples: `System.nanoTime()` in Java and `clock_gettime(CLOCK_MONOTONIC)` on Linux.

Most operating systems and languages provide both, since both are useful.

3.3 Causality and happens-before

User A sends the statement m_1 to users B and C; on receiving it, B sends the reply m_2 to A and C. Even reliable links may reorder messages, so if m_1 is delayed, C sees the reply before the message it replies to (figure 5). If m_1 creates an object in a database and m_2 updates it, a node that processes m_2 first tries to update a nonexistent object and then creates an object that is never updated. Monotonic clocks cannot help order the messages, since their timestamps are not comparable across nodes. Attaching time-of-day timestamps does not work reliably either: synchronisation always leaves some residual skew, so if A's clock is slightly ahead of B's, B may send m_2 at a time $t_2 < t_1$, where t_1 is the timestamp of m_1. Timestamp order is then inconsistent with the expected order.

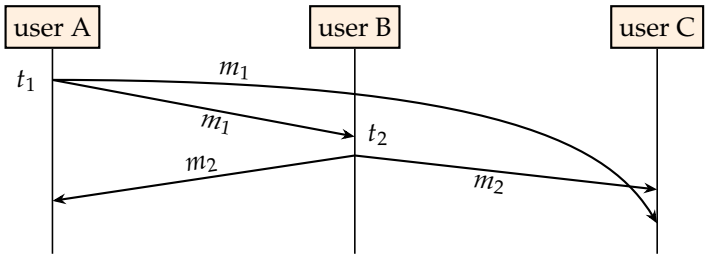


Figure 5: $m_1 = (t_1, \text{“A says: The moon is made of cheese!”})$ and $m_2 = (t_2, \text{“B says: Oh no it isn't!”})$. C receives m_2 before m_1, and even with synced clocks $t_2 < t_1$ is possible.

To define the correct order, assume each node has a single thread of execution, so there is a strict total order on the events at the same node (a multithreaded process can be modelled with one node per thread), and assume every sent message is unique (in practice, include the sender's ID and a sequence number), so a received message identifies where and when it was sent.

Definition 3.1 [Happens-before]. An event is something happening at one node: sending or receiving a message, or a local execution step. Event a happens before event b, written $a \to b$, if and only if

- a and b occurred at the same node, and a occurred before b in that node's local execution order; or
- a is the sending of some message m and b is the receipt of that same message m; or
- there exists an event c such that $a \to c$ and $c \to b$.

The relation is a partial order: possibly neither $a \to b$ nor $b \to a$. In that case a and b are concurrent, written $a \parallel b$.

The second clause rules out time travel: a message is sent before it is received. "Concurrent" does not mean "at the same time"; it means that no sequence of messages leads from one event to the other.

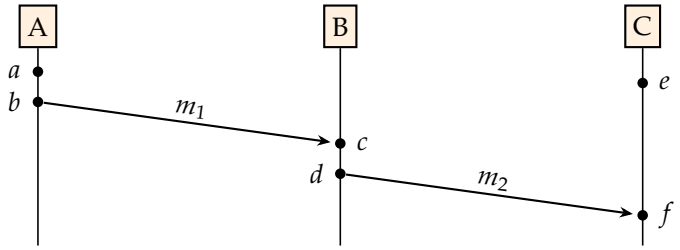


Figure 6: Happens-before example. Node execution order gives $a \to b$, $c \to d$, and $e \to f$; the messages give $b \to c$ and $d \to f$; transitivity gives $a \to c$, $a \to d$, $a \to f$, $b \to d$, $b \to f$, and $c \to f$. Finally $a \parallel e$, $b \parallel e$, $c \parallel e$, and $d \parallel e$.

Happens-before is a way of reasoning about CAUSALITY, that is, whether information could have flowed from one event to another. When $a \to b$, a might have caused b; when $a \parallel b$, a cannot have caused b. The relation thus encodes POTENTIAL CAUSALITY: what matters is not whether one event truly caused another but, for instance, that the sender of m_2 had already received m_1. The notion comes from physics, where information cannot travel faster than light. Two events a and b far apart in space but close in time cannot influence each other, since no signal from one can reach the other's location in time. An event c sufficiently close to a in space and sufficiently long after it lies within a's LIGHT CONE, so a might influence c. Messages on a network play the role of light.

Definition: Let $\prec$ be a strict total order on events. If $(a \to b) \implies (a \prec b)$, then $\prec$ is a CAUSAL ORDER (or $\prec$ is CONSISTENT WITH CAUSALITY). Note that "causal" is not "casual".

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 3 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 4: Broadcast Protocols and Logical Time

Broadcast (or multicast) protocols deliver one message to multiple recipients. They are building blocks for higher-level distributed algorithms, and they differ mainly in the order in which they deliver messages. Ordering is closely related to clocks, so we first examine how clocks can track ordering.

4.1 Logical time

Physical clocks count seconds elapsed; their timestamps are useful for many things but may be inconsistent with causality: even with NTP, if send(m_1) $\rightarrow$ send(m_2), the physical timestamp of send(m_2) may be smaller. Logical clocks instead count events and are designed to capture causal dependencies:

$$(e_1 \rightarrow e_2) \implies \big(T(e_1) < T(e_2)\big).$$

We look at two kinds, Lamport clocks and vector clocks.

Algorithm 4.1: Lamport clocks

```
1  on initialisation do
2  |  t := 0                                   // each node has its own local variable t
3  on any event occurring at the local node do
4  |  t := t + 1
5  on request to send message m do
6  |  t := t + 1
7  |  send (t, m) via the underlying network link
8  on receiving (t', m) via the underlying network link do
9  |  t := max(t, t') + 1
10 |  deliver m to the application
```

Each node keeps a counter t, incremented on every local event e; let $L(e)$ be the value of t after that increment. The current t is attached to every message sent, and a recipient moves its clock forward to the timestamp in the message (if that is greater than its counter) and then increments it. A Lamport timestamp is thus an integer counting events, with no direct relation to physical time. The algorithm assumes a crash-stop model, or crash-recovery if t is kept in stable storage (on disk).

Proposition 4.1 [Lamport timestamps]. *If $a \rightarrow b$ then $L(a) < L(b)$. However, $L(a) < L(b)$ does not imply $a \rightarrow b$, and $L(a) = L(b)$ is possible for $a \neq b$.*

The first property holds because t strictly increases from each event to the next at the same node, and the receipt of a message gets a timestamp greater than the one the message carries, so the timestamp increases along every step of a chain of $\rightarrow$. If $L(a) < L(b)$, we know only that $b \not\rightarrow a$: either $a \rightarrow b$

or $a \parallel b$. In figure 7(a), the third event on A and the first event on B both have timestamp 3.

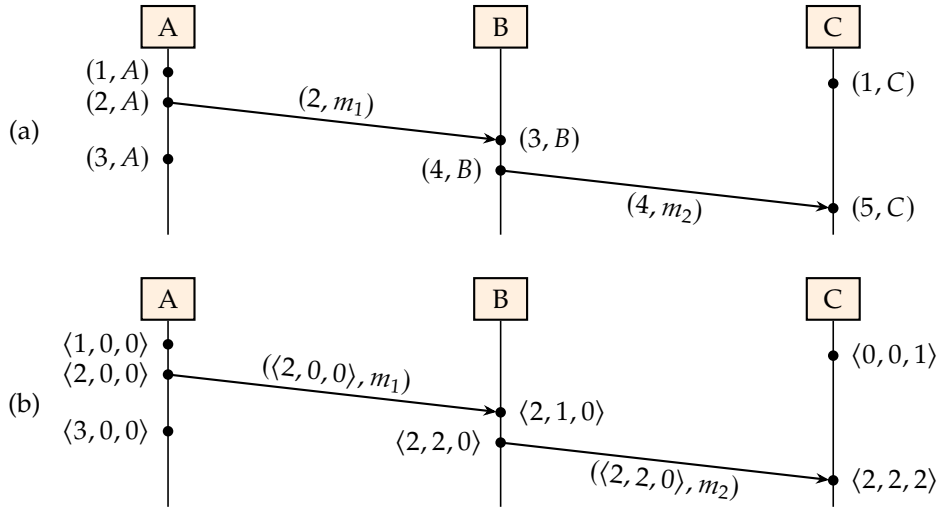


Figure 7: One execution with (a) Lamport timestamps, written $(L(e), N(e))$, and (b) vector timestamps for the vector of nodes $N = \langle A, B, C \rangle$. Each message carries the sender's current timestamp.

Let $N(e)$ be the node at which event e occurred. Assuming unique node names, the pair $(L(e), N(e))$ uniquely identifies e, and we can define a total order by comparing timestamps first and breaking ties by node name:

$$(a \prec b) \iff \big(L(a) < L(b) \ \vee \ (L(a) = L(b) \wedge N(a) < N(b))\big).$$

This order is causal: $(a \to b) \implies (a \prec b)$. In other words, $\prec$ is a linear extension of the partial order $\to$; if $a \parallel b$, the algorithm orders them arbitrarily.

Lamport timestamps cannot tell whether two events are concurrent or one happened before the other. To detect concurrency we need VECTOR CLOCKS. Put the n nodes into a vector $N = \langle N_1, N_2, \ldots, N_n \rangle$. The vector timestamp of an event a is $V(a) = \langle t_1, t_2, \ldots, t_n \rangle$, where $t_i = T[i]$ is the number of events observed by (known to have occurred at) node N_i. Each node has a current vector timestamp T; on an event at node N_i it increments its own element $T[i]$, it attaches its current vector to each message, and a recipient merges the message's vector into its own by taking the element-wise maximum and then increments its own element. (In practice the vector is often a map from node IDs to integers.)

Algorithm 4.2: Vector clocks

```
1  on initialisation at node N_i do
2  |  T := ⟨0, 0, ..., 0⟩                          // local variable at node N_i
3  on any event occurring at node N_i do
4  |  T[i] := T[i] + 1
5  on request to send message m at node N_i do
6  |  T[i] := T[i] + 1
7  |  send (T, m) via network
8  on receiving (T', m) at node N_i via the network do
9  |  T[j] := max(T[j], T'[j]) for every j ∈ {1, ..., n}
10 |  T[i] := T[i] + 1
11 |  deliver m to the application
```

In figure 7(b), when C receives m_2 from B, its entry for A also becomes 2, because the receipt depends indirectly on the two events at A; vector timestamps mirror the transitivity of happens-before. The vector timestamp of e represents the set of events $\{e\} \cup \{a \mid a \to e\}$: $\langle 2, 2, 0\rangle$ stands for the first two events of A, the first two events of B, and no events of C.

Definition: For vector timestamps in a system of n nodes:

- $T = T'$ iff $T[i] = T'[i]$ for all $i \in \{1, \ldots, n\}$;
- $T \leq T'$ iff $T[i] \leq T'[i]$ for all $i \in \{1, \ldots, n\}$;
- $T < T'$ iff $T \leq T'$ and $T \neq T'$;
- $T \parallel T'$ iff $T \not\leq T'$ and $T' \not\leq T$.

Two vectors are incomparable when each is greater in some element: for $T = \langle 2, 2, 0\rangle$ and $T' = \langle 0, 0, 1\rangle$, $T[1] > T'[1]$ but $T[3] < T'[3]$. With the set interpretation,

$$V(a) \leq V(b) \iff \big(\{a\} \cup \{e \mid e \to a\}\big) \subseteq \big(\{b\} \cup \{e \mid e \to b\}\big).$$

Proposition 4.2 [Vector timestamps]. *For all events a and b,*

$$\big(V(a) < V(b)\big) \iff (a \to b), \qquad \big(V(a) = V(b)\big) \iff (a = b), \qquad \big(V(a) \parallel V(b)\big) \iff (a \parallel b).$$

So the partial order on vector timestamps corresponds exactly to happens-before, and vector clocks compute the happens-before relation in practice: Lamport clocks give a total order consistent with causality, vector clocks capture the partial order itself. Other constructions exist, such as hybrid clocks that combine properties of logical and physical clocks.

4.2 Delivery order in broadcast protocols

Point-to-point (unicast) messaging has one recipient per message; BROADCAST generalises it to group communication, in which one node sends a message and all nodes in a group deliver it. Group membership may be fixed (static) or dynamic, and if one node is faulty the remaining members carry on. The concept is more general than IP multicast: some local networks offer hardware multicast, typically best-effort, but the Internet usually allows only unicast, so we build broadcast on point-to-point messaging. The node and timing models of Lecture 2 carry over. A broadcast may be BEST-EFFORT (it may drop messages) or RELIABLE (non-faulty nodes deliver every message, by retransmitting dropped ones), and with asynchronous or partially synchronous timing there is no upper bound on message latency.

When the application wants to send a message to the whole group, it asks a broadcast algorithm (middleware) to BROADCAST it. The algorithm SENDS messages over point-to-point links, another node RECEIVES them, and its broadcast algorithm DELIVERS the message to the application, possibly after buffering it for a while (figure 8).

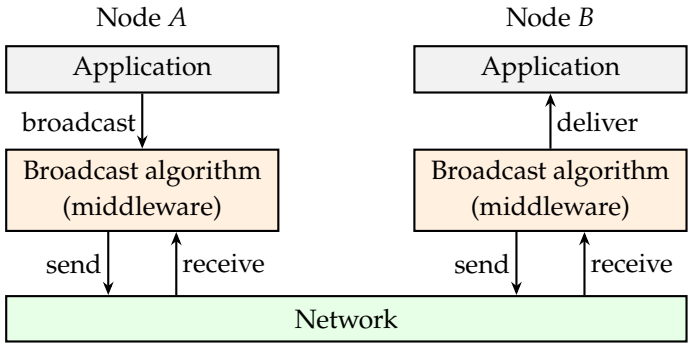


Figure 8: Receiving versus delivering. The network provides point-to-point send and receive; after receiving a message, the broadcast algorithm may queue it before delivering it to the application.

All the following forms of broadcast are reliable, with no timing guarantees; they differ in the order in which nodes may deliver messages.

Definition:

- FIFO BROADCAST: if m_1 and m_2 are broadcast by the same node and broadcast(m_1) $\rightarrow$ broadcast(m_2), then m_1 must be delivered before m_2.
- CAUSAL BROADCAST: if broadcast(m_1) $\rightarrow$ broadcast(m_2), then m_1 must be delivered before m_2.
- TOTAL ORDER BROADCAST (also ATOMIC BROADCAST): if m_1 is delivered before m_2 on one node, then m_1 must be delivered before m_2 on all nodes.
- FIFO-TOTAL ORDER BROADCAST: the combination of FIFO broadcast and total order broadcast.

We assume a node that broadcasts a message also delivers it to itself (the loops in figure 9); this

matters for total order broadcast.

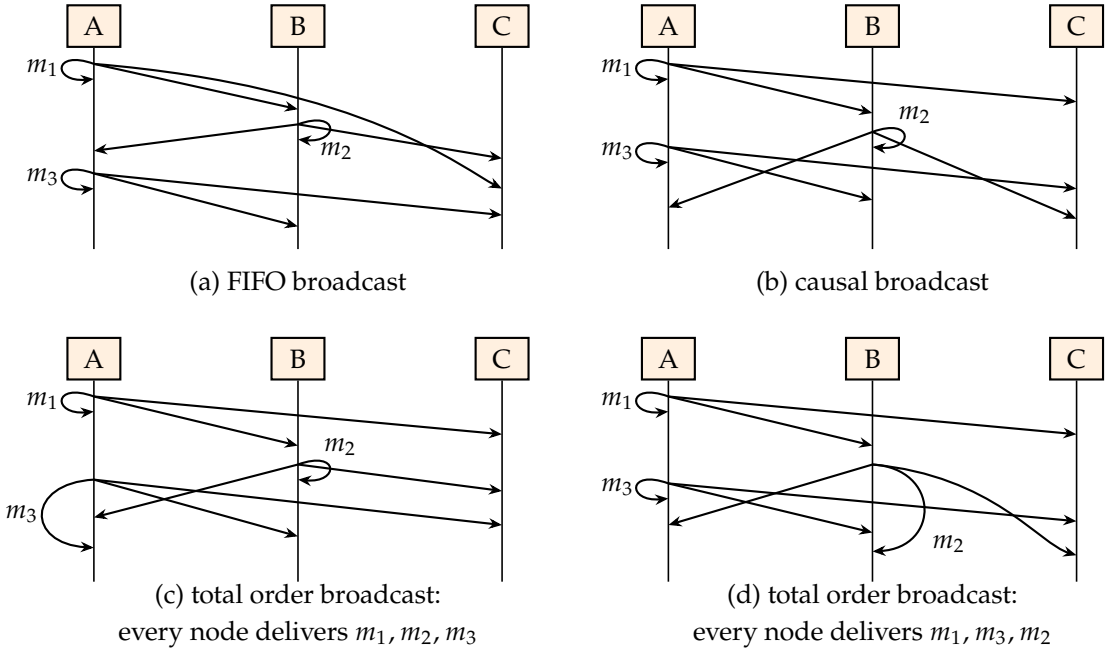


Figure 9: Example executions. Each node broadcasts where its message is labelled; a loop is the delivery of a node's own message, and the other arrows are receipts at the other nodes. In (a) and (b) nodes deliver in different orders; in (c), A's delivery of m_3 to itself, and in (d), B's delivery of m_2 to itself, must wait.

FIFO broadcast is the weakest: messages sent by the same node are delivered in the order they were sent, and messages from different nodes in any order. In figure 9(a), m_1 must be delivered before m_3 (both from A), but m_2 may come before, between, or after them; the valid orders are (m_2, m_1, m_3), (m_1, m_2, m_3), and (m_1, m_3, m_2). This execution is valid FIFO broadcast but violates causality, since C delivers m_2 before m_1 although B broadcast m_2 after delivering m_1.

Causal broadcast delivers causally related messages in causal order and concurrent messages in any order. In (a), causal broadcast at C would have to HOLD BACK (buffer) m_2 until after delivering m_1. In (b), broadcast$(m_1) \rightarrow$ broadcast(m_2) and broadcast$(m_1) \rightarrow$ broadcast(m_3), while m_2 and m_3 are concurrent, so the valid orders are (m_1, m_2, m_3) and (m_1, m_3, m_2): A and C deliver m_1, m_3, m_2 and B delivers m_1, m_2, m_3.

FIFO and causal broadcast let nodes deliver in different orders; total order broadcast forces all nodes, including a node's deliveries to itself, to deliver in the same order, though which order is not specified. In (c) every node delivers m_1, m_2, m_3 and in (d) every node delivers m_1, m_3, m_2. Nodes may again need to hold back messages: if m_3 must precede m_2 but C receives m_2 first, C holds m_2 back until m_3 arrives. Moreover, in FIFO and causal broadcast a node can deliver its own message immediately, without communicating, whereas in total order broadcast it may have to wait: in (c), A delivers m_3 to itself only after receiving m_2 from B, and in (d), B's self-delivery of m_2 waits for m_3. FIFO-total order broadcast additionally requires messages from the same node to be delivered in the order they were sent; (c) and (d) satisfy it, since m_1 precedes m_3 in both.

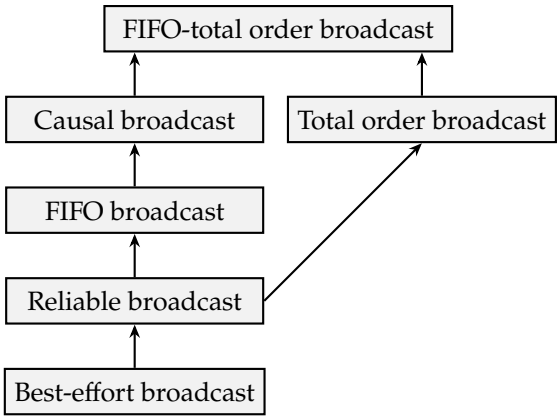


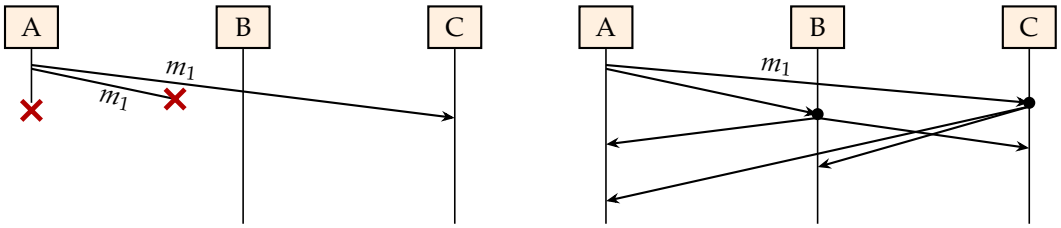
Figure 10: Relationships between broadcast models. An arrow points from a model to a strictly stronger one.

Figure 10 arranges the models in a hierarchy. For example, FIFO-total order broadcast is strictly stronger than causal broadcast: every valid FIFO-total order broadcast protocol is also a valid causal broadcast protocol, but not conversely.

4.3 Broadcast algorithms

Implementing broadcast has two layers: first make best-effort broadcast reliable by retransmitting dropped messages, so that every node receives every message; then enforce the delivery order on top of reliable broadcast.

A first attempt is for the broadcasting node to send the message directly to every other node over reliable links (retry and deduplicate). But a message may be dropped and the sender may crash before retransmitting it, and then some node never receives it (figure 11a). In EAGER RELIABLE BROADCAST, the first time a node receives a particular message, it re-broadcasts it to each other node via reliable links (figure 11b). Even if some nodes crash, all remaining non-faulty nodes receive every message. It is inefficient, though: without faults, each node receives every message $n - 1$ times, so a message is sent $O(n^2)$ times in a group of n nodes.



(a) direct sends (b) eager reliable broadcast

Figure 11: (a) Sending directly to every node: the message to B is lost and A crashes before retrying, so B never receives m_1. (b) Eager reliable broadcast of m_1: dots mark the first receipt at B and C, after which each forwards m_1 to the other nodes.

Many variants optimise fault tolerance, the time until all nodes receive a message, and the bandwidth used. A common family is GOSSIP PROTOCOLS (EPIDEMIC PROTOCOLS), useful when broadcasting to a large number of nodes. A node sends a message to a small fixed number of randomly chosen nodes (e.g. 3), and a node receiving a message for the first time forwards it to that many randomly chosen nodes, like gossip, rumours, or an infection spreading through a population. Delivery to all nodes is not strictly guaranteed, since some node might always be omitted by the random choices, but with appropriate parameters the message eventually reaches all nodes with high probability, and the protocol is very resilient to message loss and crashes while remaining efficient. On top of reliable broadcast (eager or gossip) we build the ordered forms.

Algorithm 4.3: FIFO broadcast

```
1  on initialisation do
2  |  sendSeq := 0; delivered := ⟨0, 0, ..., 0⟩; buffer := {}
3  on request to broadcast m at node N_i do
4  |  send (i, sendSeq, m) via reliable broadcast
5  |  sendSeq := sendSeq + 1
6  on receiving msg from reliable broadcast at node N_i do
7  |  buffer := buffer ∪ {msg}
8  |  while ∃ sender, m. (sender, delivered[sender], m) ∈ buffer do
9  |  |  deliver m to the application
10 |  |  delivered[sender] := delivered[sender] + 1
```

Each message from N_i is tagged with i and a sequence number (0 for its first message, 1 for the second, and so on). The state is *sendSeq* (messages broadcast by this node), *delivered* (per sender, the messages delivered from it), and *buffer* (messages held back). The loop delivers any buffered message carrying the next sequence number expected from its sender, so each sender's messages are delivered in order.

The causal broadcast algorithm (algorithm 4.4) attaches a vector instead of a sequence number. It is sometimes called a vector clock algorithm, although its vector counts the messages delivered from each sender rather than the events at each node. The vector *deps* copies *delivered*, so every message delivered locally before this broadcast must precede it in the causal order; setting the sender's own element to *sendSeq* makes each message depend on the previous message from the same node. On receipt, a message is causally ready when $deps \leq delivered$ (the vector order above), meaning this node has delivered every message that must precede it; ready messages are delivered, removed from the buffer, and counted.

Algorithm 4.4: Causal broadcast

```
1  on initialisation do
2      sendSeq := 0; delivered := ⟨0, 0, ..., 0⟩; buffer := {}
3  on request to broadcast m at node N_i do
4      deps := delivered; deps[i] := sendSeq
5      send (i, deps, m) via reliable broadcast
6      sendSeq := sendSeq + 1
7  on receiving msg from reliable broadcast at node N_i do
8      buffer := buffer ∪ {msg}
9      while ∃(sender, deps, m) ∈ buffer. deps ≤ delivered do
10         deliver m to the application
11         buffer := buffer \ {(sender, deps, m)}
12         delivered[sender] := delivered[sender] + 1
```

Total order broadcast (and FIFO-total order broadcast) is trickier. Two simple approaches:

- *Single leader:* one node is designated the leader (SEQUENCER); to broadcast a message, send it to the leader, which broadcasts it via FIFO broadcast. If the leader crashes, no more messages are delivered, and changing the leader safely is difficult.
- *Lamport clocks:* attach a Lamport timestamp to every message and deliver messages in the total order of timestamps. The problem is knowing when all messages with timestamp less than T have been seen: we need FIFO links and must wait for a message with timestamp at least T from every node.

Neither is fault tolerant: the crash of a single node can stop all other nodes from delivering messages (in the first approach the leader is a single point of failure). Fault-tolerant total order broadcast is the subject of Lecture 6.

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 4 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 5: Replication

Definition: Replication means keeping a copy of the same data on multiple nodes, each of which is called a replica.

Replication is a standard feature of distributed databases, filesystems, caches, and other storage systems, and one of the main mechanisms for fault tolerance: if some replicas are faulty, the others are still accessible. It also spreads load across replicas. Compared with RAID (Redundant Array of Independent Disks), which replicates within one computer under a single controller, the nodes of a distributed system act independently, and replicas can be placed around the world, near users. Replicating data that never changes is easy (just copy it), so the problem is managing changes.

5.1 Manipulating remote state

Consider "liking" a post on a social network: the database on the network's servers stores who liked it and how many likes there are; we call the data in a database its state. A client sends "increment post.likes"; if the acknowledgement is lost, the client retries, and the counter is incremented twice (12,300 likes become 12,302). A Twitter profile once showed a user following −20 people, plausibly because the decrement of a follow counter was retried during unfollowing; deducting £1,000 from a bank balance is the same database operation. One remedy is to deduplicate requests, which requires the database to track which requests it has already seen, in stable storage under a crash-recovery model (storing the requests or metadata such as a vector clock). The alternative is to make requests idempotent.

Definition: A function f is idempotent if $f(x) = f(f(x))$.

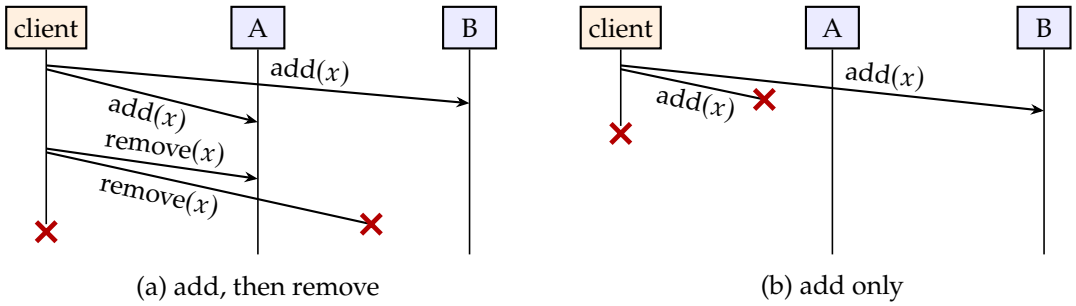
Incrementing a counter, $f(likeCount) = likeCount + 1$, is not idempotent, but adding an element to a set, $f(likeSet) = likeSet \cup \{userID\}$, is. So it may be better to store the set of users who liked a post and derive the count as its cardinality. Idempotent requests can be retried without deduplication. The choices of retry behaviour are:

- at-most-once semantics: send the request and do not retry, so the update may not happen;
- at-least-once semantics: retry the request until acknowledged, so the update may be repeated;
- exactly-once semantics: retry plus idempotence or deduplication, so the update may be applied several times but with the same effect as once.

Idempotence is very useful in practice, especially for RPC, where retries are often unavoidable.

Idempotence is not enough when several updates are in progress. Client 1 adds its user ID to the likes, $f(likes) = likes \cup \{userID\}$, but the acknowledgement is lost. Client 2 reads the likes

(including the ID) and removes it, $g(likes) = likes \setminus \{userID\}$. Client 1 then retries and adds the ID again, although the removal happened causally after the addition. Here $f(f(x)) = f(x)$, but $f(g(f(x))) \neq g(f(x))$. A similar problem arises with two replicas (figure 12): the outcome (x absent from A, present on B) is the same in two scenarios with opposite intentions, so when the replicas reconcile, they cannot tell whether x should be on both or on neither.



(a) add, then remove (b) add only

Figure 12: (a) A client adds x to both replicas, then removes it, but the remove request to B is lost and the client crashes. (b) A client adds x, the request to A is lost, and the client crashes. Both end with $x \notin A$ and $x \in B$.

The solution has two parts. First, attach a logical timestamp to every update and store it in the database as part of the written data, so every record has the timestamp of its last write. Second, do not actually delete a record: "remove(x)" writes a special update, a TOMBSTONE, labelling x with false to mark it as invisible (figure 13).

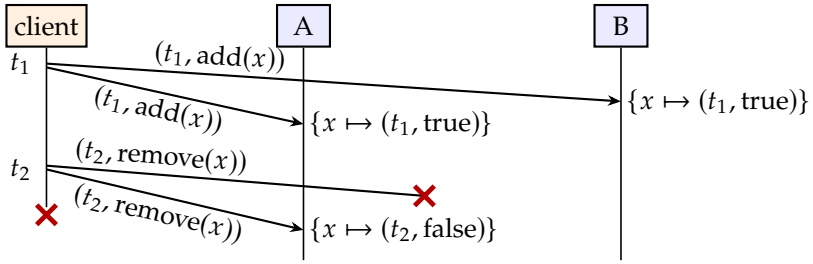


Figure 13: Timestamps and tombstones. The client's remove request to B is lost, so A holds the tombstone (t_2, false) for x while B still holds (t_1, true).

Many replicated systems run a protocol in which replicas periodically communicate to detect and reconcile differences, called ANTI-ENTROPY, so that they eventually hold consistent copies. For a given key, the record with the latest timestamp is propagated and records with earlier timestamps are discarded: in figure 13, A has $\{x \mapsto (t_2, \text{false})\}$ and B has $\{x \mapsto (t_1, \text{true})\}$, and since $t_1 < t_2$ both end with $\{x \mapsto (t_2, \text{false})\}$. Tombstones distinguish a deleted record from one not yet created, and timestamps tell which version is newer. Timestamps also fix the retry problem above: a retried request carries the timestamp of the original request, so it does not overwrite a value written by a causally later request with a greater timestamp.

Timestamps also handle concurrent updates. Client 1 sets key x to v_1 with timestamp t_1 while

client 2 concurrently sets x to v_2 with timestamp t_2; replica A receives v_2 first and B receives v_1 first (figure 14). To end in the same state, replicas rely on the order of the timestamps, not the order of arrival.

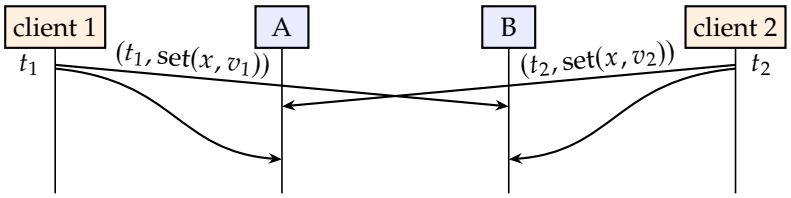


Figure 14: Concurrent writes by different clients arrive at replicas A and B in opposite orders.

There are two common approaches.

- Last writer wins (LWW): use timestamps with a total order (e.g. Lamport clocks); keep v_2 and discard v_1 if $t_2 > t_1$. Concurrent updates are ordered arbitrarily and all but the greatest timestamp are discarded, so this implies data loss, which some applications accept.
- Multi-value register: use timestamps with a partial order (e.g. vector clocks); v_2 replaces v_1 if $t_2 > t_1$, but both $\{v_1, v_2\}$ are preserved if $t_1 \parallel t_2$. Concurrently written values are called conflicts or siblings, and the application can later merge them into a single value.

Vector clocks can become expensive: every client needs an entry, so with many clients (or clients that take a new identity on every restart) the vectors may take more space than the data. Further logical clocks, such as dotted version vectors, optimise this case.

5.2 Quorums

A replica may be unavailable because of a node fault (crash, hardware problem), a network partition, or planned maintenance. Without fault tolerance, more replicas make reliability worse, since the probability that some replica is faulty grows; with fault tolerance it improves, since all replicas being faulty at once is much less likely. Assume each replica is faulty or unavailable with probability p, independently (not actually true, but an acceptable approximation). Then all n replicas are faulty with probability p^n, and at least one is faulty with probability $1 - (1-p)^n$. For $p = 0.01$:

Replicas n	$P(\geq 1$ faulty)	$P(\geq \frac{n+1}{2}$ faulty)	$P(\text{all } n \text{ faulty})$
1	0.01	0.01	0.01
3	0.03	$3 \cdot 10^{-4}$	10^{-6}
5	0.049	$1 \cdot 10^{-5}$	10^{-10}
100	0.63	$6 \cdot 10^{-74}$	10^{-200}

Suppose two replicas A and B both hold $x = v_0$ with timestamp t_0. A client writes $(t_1, \text{set}(x, v_1))$, which succeeds on B but fails on A (A is temporarily unavailable); it then reads x, which succeeds

on A but fails on B, and gets the initial value v_0. From the client's point of view its write has been lost (like refreshing a page and not seeing the update you just posted).

Definition: Read-after-write consistency (or read-your-writes consistency) requires that after a client writes a value, the same client reads either the value it wrote or a later value (another client may have overwritten it concurrently).

Writing to and reading from both replicas would guarantee it, but then a single unavailable replica blocks all writes or reads. Instead use three replicas, send every request to all of them, and consider it successful once at least 2 respond (figure 15). Here the write succeeds on B and C and the read on A and B, and with "2 out of 3" for both, at least one response to the read (B's) comes from a replica that saw the latest write. The read returns (t_0, v_0) from A and (t_1, v_1) from B, and the client uses the timestamps to return the newer v_1.

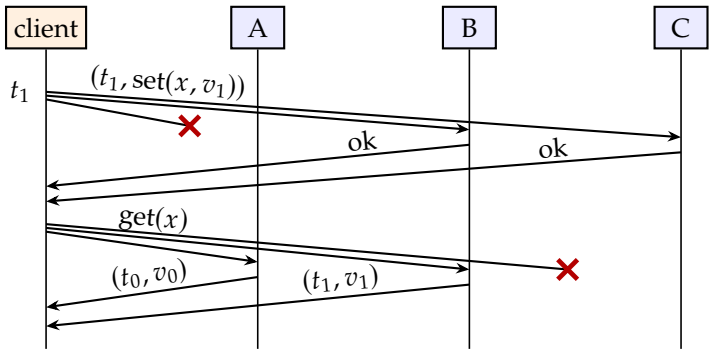


Figure 15: Quorum (2 out of 3): the write succeeds on B and C, the read on A and B, and the client chooses between (t_0, v_0) and (t_1, v_1) by timestamp.

Definition: A quorum is a minimum set of nodes that must respond to a request for it to succeed (in politics, the minimum number of votes for a valid decision). In figure 15, $\{B, C\}$ is a write quorum and $\{A, B\}$ a read quorum. A majority quorum is any subset of strictly more than half of the nodes.

For read-after-write consistency, the write quorum and the read quorum must intersect, so that the read quorum contains a node that acknowledged the write. With nodes $\{A, B, C\}$ the majority quorums are $\{A, B\}$, $\{A, C\}$, and $\{B, C\}$. For odd n any subset of size $\frac{n+1}{2}$ is a majority quorum (2 of 3, 3 of 5, ...); for even n the size is $\left\lceil \frac{n+1}{2} \right\rceil = \frac{n+2}{2}$ (3 of 4). Any two majority quorums have at least one element in common; other quorum constructions are also possible.

Proposition 5.1 [Read and write quorums]. *In a system with n replicas, if a write is acknowledged by w replicas (a write quorum), a subsequent read gets responses from r replicas (a read quorum), and $r + w > n$, then the read sees the previously written value (or a value that subsequently overwrote it).*

Indeed, if the two quorums were disjoint they would contain $r + w > n$ distinct replicas, so they share at least one replica. A typical choice is $r = w = \frac{n+1}{2}$ for $n = 3, 5, 7, \ldots$ (majorities); for example, with five replicas A to E, the read quorum $\{A, B, C\}$ and the write quorum $\{C, D, E\}$ share C. Reads

tolerate $n - r$ unavailable replicas and writes $n - w$, so with majority quorums 3 replicas tolerate 1 unavailable replica and 5 tolerate 2.

Some updates may be missing from some replicas at any moment (in figure 15, (t_1, v_1) is missing from A). Besides anti-entropy, clients can help disseminate updates by READ REPAIR (figure 16): having read (t_1, v_1) from B, an older (t_0, v_0) from A, and nothing from C, the client sends the update with its original timestamp t_1 (it is a retry of a previous update, not a new one) to A, and also to C, wasting only a little bandwidth if C already has it. Any client can perform read repair on any read, whether or not it made the original update. Databases replicated this way are called DYNAMO-STYLE, after Amazon's Dynamo, which popularised the model, although the approach predates it.

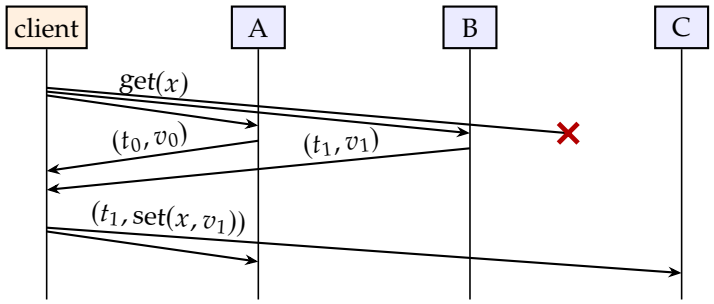


Figure 16: Read repair: since $t_0 < t_1$, the update (t_1, v_1) is more recent, and the client propagates it to the other replicas.

5.3 Replication using broadcast

The quorum approach essentially uses best-effort broadcast: a client broadcasts each request to all replicas, without reliability or ordering guarantees. Stronger broadcasts give other replication schemes; total order broadcast makes every node deliver the same messages in the same order.

Definition: In STATE MACHINE REPLICATION (SMR), every update is broadcast to all replicas by FIFO-total order broadcast, and a replica that delivers an update message applies it to its own state. Applying an update must be deterministic. A replica is then a STATE MACHINE: it starts in a fixed initial state and goes through the same sequence of state transitions in the same order as every other replica, so all replicas end up in the same state.

Algorithm 5.1: State machine replication

```
1 on request to perform update u do
2 |   send u via FIFO-total order broadcast
3 on delivering u through FIFO-total order broadcast do
4 |   update state using arbitrary deterministic logic!
```

Determinism means any two replicas in the same state given the same input end up in the same next state; even errors must be deterministic, since an update that succeeds on one replica but fails on another makes them inconsistent. Otherwise the transition logic can be arbitrarily complex and

depend on both the message and the current state: some distributed databases execute an entire transaction with arbitrary business logic on each replica (ACTIVE REPLICATION). Closely related ideas are serializable transactions (executed in delivery order) and blockchains, distributed ledgers, and smart contracts: the chain of blocks is the sequence of messages delivered by a total order broadcast, each replica deterministically executes the transactions in the blocks to determine the state of the ledger (who owns which money), and a smart contract is a deterministic program a replica runs when a particular message is delivered. The limitations are those of total order broadcast: a replica cannot update its state immediately but must wait until the update is delivered back to it, and fault tolerance requires fault-tolerant total order broadcast (Lecture 6).

Total order broadcast via a leader is also how many databases replicate (figure 17). One replica is designated the LEADER (primary, master), and transactions that modify the database must execute on it. The leader may execute several transactions concurrently but commits them in a total order, broadcasting each transaction's data changes to the FOLLOWER replicas, which apply them in commit order. This PASSIVE REPLICATION (PRIMARY-BACKUP REPLICATION) is equivalent to total order broadcast of transaction commit records.

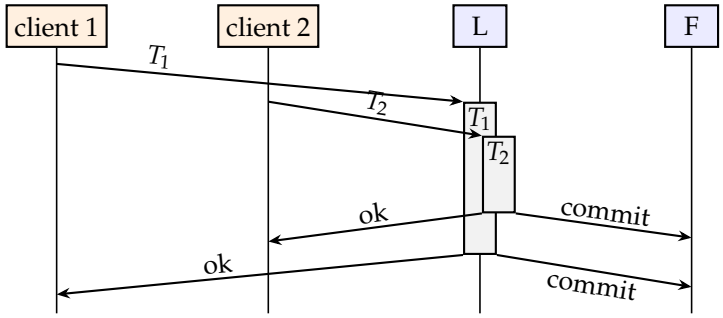


Figure 17: Database leader replica. The leader L executes T_1 and T_2 concurrently but commits T_2 first; follower F applies the transaction log in commit order.

Weaker broadcasts can also be used for replication, but determinism is no longer sufficient. Causal broadcast delivers causally related updates in the same order everywhere but concurrent updates in any order, so replicas end up in the same state only if concurrent updates are COMMUTATIVE: $f(g(x)) = g(f(x))$, so the result does not depend on the order of application. In general:

Broadcast	Assumptions about the state update function
total order	deterministic (SMR)
causal	deterministic, concurrent updates commute
reliable	deterministic, all updates commute
best-effort	deterministic, commutative, idempotent, tolerates message loss

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 5 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 6: Consensus

Total order broadcast enables state machine replication, and it can be implemented by routing all messages through a single leader that distributes them by FIFO broadcast. But the leader is a single point of failure: if it becomes unavailable, the whole system grinds to a halt. In MANUAL FAILOVER, a human operator chooses a new leader and reconfigures every node to use it. Many databases do this, and it is fine for planned maintenance (such as rebooting the leader to install updates), but for an unplanned outage (a crash, a hardware failure, a network problem) humans are slow, and even in the best case the system cannot process updates for several minutes. Consensus algorithms transfer leadership automatically.

6.1 Introduction to consensus

Definition: In the CONSENSUS problem, several nodes want to come to agreement about a value. One or more nodes propose a value, and the algorithm decides on one of the proposed values. All nodes decide on the same value (except faulty nodes, which may not decide anything), and the decision is final: a node never changes its mind once it has decided.

In the context of total order broadcast, the value to agree on is the next message to deliver, and once one node decides on a message order, all nodes decide the same order. Consensus and total order broadcast are formally equivalent. To turn total order broadcast into consensus, a node proposes a value by broadcasting it, and the first message delivered is the decision. To turn consensus into total order broadcast, run a separate instance of consensus to decide the first, second, third, ... message to deliver; a node that wants to broadcast proposes its message for one of these rounds. The best-known algorithms are PAXOS, which provides consensus on a single value, and its generalisation MULTI-PAXOS, which provides FIFO-total order broadcast; RAFT, Viewstamped Replication, and Zab provide FIFO-total order broadcast by default.

Paxos and Raft assume a partially synchronous, crash-recovery system model with fair-loss links. The node and network assumptions can be weakened to Byzantine, and such algorithms are used in blockchains, but they are significantly more complicated and less efficient; crash-recovery algorithms suffice in many practical settings, such as datacenters with trusted private networks. Partial synchrony, however, cannot be weakened to asynchrony.

Theorem 6.1 [FLP RESULT**].** *There is no deterministic consensus algorithm that is guaranteed to terminate in an asynchronous crash-stop system model.*

The result, named after Fischer, Lynch, and Paterson, is one of the most important theorems of distributed computing. Consensus needs a failure detector, which needs a local clock to trigger timeouts; without clocks a deterministic algorithm might never terminate. Randomised

(nondeterministic) algorithms can get around the result, but most practical systems instead use clocks for timeouts. Since partial synchrony bounds neither latency nor execution speed, consensus algorithms must guarantee their SAFETY properties (every node decides on the same messages in the same order) regardless of timing, even if messages are arbitrarily delayed. Paxos and Raft use clocks only for timeouts and the failure detector, so only LIVENESS (a message is eventually delivered) depends on timing.

At the core of most consensus algorithms is LEADER ELECTION. Multi-Paxos, Raft, and similar algorithms use a leader to sequence messages. A failure detector (a timeout, typically after no message from the leader for some time) determines that the leader is suspected to have crashed or become unavailable, and the nodes then elect a new one: a CANDIDATE asks the others to vote for it, and becomes leader if a quorum votes in favour. With majority quorums the vote succeeds as long as a majority of nodes (2 of 3, 3 of 5) are working and can communicate. Two leaders at the same time (SPLIT BRAIN) could make inconsistent decisions and violate the safety of total order broadcast. Raft captures "at any one time" with a TERM, an integer incremented every time a leader election starts, and ensures at most one leader per term:

- the term is incremented every time a leader election is started;
- a node can only vote once per term;
- a quorum of nodes is required to elect a leader in a term.

With nodes A to E, if A, B, and C elect a leader, then C, D, and E cannot elect a different leader in the same term, because C has already voted. Different terms may have different leaders.

Multiple leaders from different terms cannot be prevented, because a timeout-based failure detector may be inaccurate in a partially synchronous system. If node 1 is leader in term t but a network partition cuts it off from nodes 2 and 3, they may elect a new leader in term $t + 1$ although node 1 is still running, and node 1 may not even know that a new leader has been elected. A leader therefore must not act unilaterally: for every decision (message to deliver), it first gets acknowledgements from a quorum (figure 18). In the first round trip, the leader is elected by the votes of the other nodes. In the second, it proposes the next message to deliver, and the followers acknowledge that they know of no leader with a term later than t. Finally, the leader delivers m and tells the followers to do the same. If another leader has been elected, at least one node in the second-round quorum also voted for the new leader, so the old leader finds out and can no longer decide on further messages: several leaders may exist at once, but the algorithm stays safe.

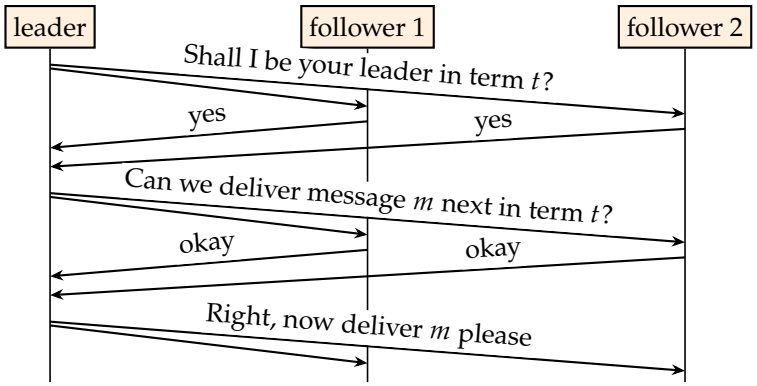


Figure 18: Checking whether a leader has been voted out: after its election, the leader gets acknowledgements from a quorum before delivering each message.

6.2 The Raft consensus algorithm

Raft provides fault-tolerant FIFO-total order broadcast. A node is in one of three states (figure 19). It starts as a FOLLOWER when it first starts running or recovers from a crash, and awaits messages. If it receives no message from a leader or candidate for some time, it suspects the leader has failed, becomes a CANDIDATE, increments the term, and starts an election in that term. The timeout for detecting leader failure is randomised, to reduce the probability of several nodes becoming candidates at once and competing. A candidate that hears from another candidate or leader with a higher term returns to the follower state; one that receives votes from a quorum becomes LEADER; and if not enough votes arrive within some time, the election times out and restarts with a higher term. A leader remains leader until it shuts down or crashes, or until it receives a message with a higher term (e.g. after a partition led another node to start an election), in which case it steps down to follower.

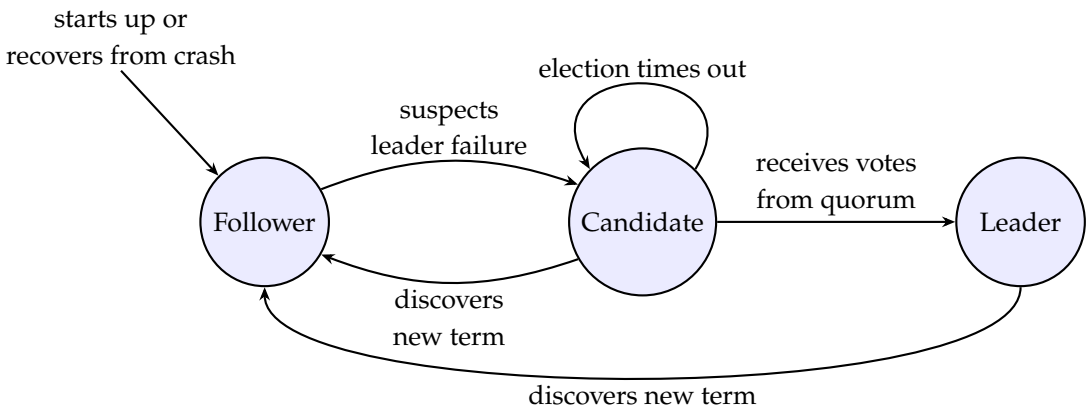


Figure 19: Node state transitions in Raft.

The variables in the initialisation block of algorithm 6.1 form a node's state. Four of them (*currentTerm*, *votedFor*, *log*, *commitLength*) are kept in stable storage, since they must survive a crash;

the others are volatile, and crash recovery resets them. Each node has a unique ID, and a global constant *nodes* contains the IDs of all nodes (this version does not handle reconfiguration). The *log* is an array of entries with properties *msg*, the message to be delivered by total order broadcast, and *term*, the term in which it was broadcast. It is indexed from zero, so a log of three messages broadcast in term 1 is $\langle (m_1, 1), (m_2, 1), (m_3, 1) \rangle$ with entries $log[0], log[1], log[2]$. Raft replicates the log across nodes by appending entries at the end. When an entry and all its predecessors have been replicated to a quorum, it is COMMITTED, and its *msg* is delivered to the application at that moment. An entry may change before it is committed, but once committed it is final, and all nodes commit the same sequence of entries; delivering committed entries in log order therefore gives FIFO-total order broadcast.

To start an election, a node increments *currentTerm*, becomes a candidate, votes for itself, and sends each node a VoteRequest containing its ID, its new term, the length of its log, and the term of its last log entry.

Algorithm 6.1: Raft (1/9): initialisation and starting an election

```
1  on initialisation do
2      currentTerm := 0; votedFor := null
3      log := ⟨⟩; commitLength := 0
4      currentRole := follower; currentLeader := null
5      votesReceived := {}; sentLength := ⟨⟩; ackedLength := ⟨⟩
6  on recovery from crash do
7      currentRole := follower; currentLeader := null
8      votesReceived := {}; sentLength := ⟨⟩; ackedLength := ⟨⟩
9  on node nodeId suspects leader has failed, or on election timeout do
10     currentTerm := currentTerm + 1; currentRole := candidate
11     votedFor := nodeId; votesReceived := {nodeId}; lastTerm := 0
12     if log.length > 0 then lastTerm := log[log.length − 1].term
13     msg := (VoteRequest, nodeId, currentTerm, log.length, lastTerm)
14     for each node ∈ nodes do send msg to node
15     start election timer
```

A node receiving a VoteRequest (algorithm 6.2; the prefix *c* stands for candidate) first becomes a follower in the candidate's term if that term is greater than its own, even if it was a leader in an earlier term. It then checks that the candidate's log is at least as up-to-date as its own, so that a candidate with an outdated log, which could lose committed entries, cannot become leader: the candidate's log is acceptable if its last entry has a higher term than the recipient's last entry, or the same term and the candidate's log is at least as long (*logOk*). The variable *votedFor* records any vote already cast in *currentTerm*. If the candidate's term is current, its log is ok, and the node has not voted for another candidate in this term, it votes for the candidate; otherwise it refuses. The response carries the voter's ID and the term of the vote.

Algorithm 6.2: Raft (2/9): voting on a new leader

```
1  on receiving (VoteRequest, cId, cTerm, cLogLength, cLogTerm) at node nodeId do
2      if cTerm > currentTerm then
3          currentTerm := cTerm; currentRole := follower
4          votedFor := null
5      lastTerm := 0
6      if log.length > 0 then lastTerm := log[log.length − 1].term
7      logOk := (cLogTerm > lastTerm) ∨ (cLogTerm = lastTerm ∧ cLogLength ≥ log.length)
8      if cTerm = currentTerm ∧ logOk ∧ votedFor ∈ {cId, null} then
9          votedFor := cId
10         send (VoteResponse, nodeId, currentTerm, true) to node cId
11     else
12         send (VoteResponse, nodeId, currentTerm, false) to node cId
```

The candidate (algorithm 6.3) ignores responses for earlier terms, which may arrive late. If a response carries a higher term, it cancels the election and becomes a follower. If the term is current and the vote is granted, it adds the voter to *votesReceived*, and once these form a quorum it becomes leader. On the leader only, *sentLength* and *ackedLength* map each follower F to the number of log entries (counted from the start) sent to F and acknowledged by F. A new leader assumes each follower has been sent the whole log ($sentLength[F] := log.\text{length}$) and that nothing has been acknowledged; if the first assumption is wrong it is corrected later (algorithm 6.8). It then calls ReplicateLog for each follower, which informs them of the new leader.

Algorithm 6.3: Raft (3/9): collecting votes

```
1  on receiving (VoteResponse, voterId, term, granted) at nodeId do
2      if currentRole = candidate ∧ term = currentTerm ∧ granted then
3          votesReceived := votesReceived ∪ {voterId}
4          if |votesReceived| ≥ ⌈(|nodes| + 1)/2⌉ then
5              currentRole := leader; currentLeader := nodeId
6              cancel election timer
7              for each follower ∈ nodes \ {nodeId} do
8                  sentLength[follower] := log.length
9                  ackedLength[follower] := 0
10                 ReplicateLog(nodeId, follower)
11     else if term > currentTerm then
12         currentTerm := term
13         currentRole := follower
14         votedFor := null
15         cancel election timer
```

To broadcast a message (algorithm 6.4), a leader appends an entry to its own log, records that it has acknowledged its own entry, and calls ReplicateLog for every other node; any other node forwards the request to the leader via a FIFO link, which preserves FIFO-total order. The leader also calls ReplicateLog periodically even with no new message: this tells followers that the leader is alive, retransmits lost messages, and updates followers about which messages can be committed.

Algorithm 6.4: Raft (4/9): broadcasting messages

```
1  on request to broadcast msg at node nodeId do
2      if currentRole = leader then
3          append the record (msg : msg, term : currentTerm) to log
4          ackedLength[nodeId] := log.length
5          for each follower ∈ nodes \ {nodeId} do
6              ReplicateLog(nodeId, follower)
7      else
8          forward the request to currentLeader via a FIFO link
9  periodically at node nodeId do
10     if currentRole = leader then
11         for each follower ∈ nodes \ {nodeId} do
12             ReplicateLog(nodeId, follower)
```

The function ReplicateLog (algorithm 6.5) sends the follower the *suffix* of the log starting at index *sentLength*[*followerId*], that is, the entries not yet sent (empty if *sentLength*[*followerId*] = *log*.length), in a LogRequest with the leader's ID, its current term, the length *prefixLen* of the log prefix preceding the suffix, the term of the last entry of that prefix, and *commitLength*, the number of committed entries counted from the start of the log. A LogRequest with an empty suffix serves as a heartbeat.

Algorithm 6.5: Raft (5/9): replicating from leader to followers

```
1  function ReplicateLog(leaderId, followerId)
2      prefixLen := sentLength[followerId]
3      suffix := ⟨log[prefixLen], log[prefixLen + 1], . . . , log[log.length − 1]⟩
4      prefixTerm := 0
5      if prefixLen > 0 then
6          prefixTerm := log[prefixLen − 1].term
7      send (LogRequest, leaderId, currentTerm, prefixLen, prefixTerm, commitLength, suffix) to
        followerId
```

A follower receiving a LogRequest (algorithm 6.6) adopts a later term and accepts the sender as leader; a candidate in the same term also becomes a follower and recognises the sender as leader. It then checks that its log is consistent with the leader's: it must contain at least *prefixLen* entries, and the last entry of that prefix must have the same term as on the leader. Raft ensures that if two nodes have the same term at the same log index, their logs are identical up to and including that index, so *logOk* means the follower's first *prefixLen* entries equal the leader's. If the term is current and *logOk* holds, the follower appends the suffix and acknowledges the number of entries received; otherwise it replies with an error.

Algorithm 6.6: Raft (6/9): followers receiving messages

```
1  on receiving (LogRequest, leaderId, term, prefixLen, prefixTerm, leaderCommit, suffix) at node
     nodeId do
2      if term > currentTerm then
3          currentTerm := term; votedFor := null
4          cancel election timer
5      if term = currentTerm then
6          currentRole := follower; currentLeader := leaderId
7      logOk := (log.length ≥ prefixLen) ∧ (prefixLen = 0 ∨ log[prefixLen − 1].term = prefixTerm)
8      if term = currentTerm ∧ logOk then
9          AppendEntries(prefixLen, leaderCommit, suffix)
10         ack := prefixLen + suffix.length
11         send (LogResponse, nodeId, currentTerm, ack, true) to leaderId
12     else
13         send (LogResponse, nodeId, currentTerm, 0, false) to leaderId
```

AppendEntries (algorithm 6.7) extends the follower's log. If the follower already has entries at *log*[*prefixLen*] and beyond, it compares the terms at the last index present in both its log and the suffix; if they differ, the existing entries came from a superseded leader, and the log is truncated to its first *prefixLen* entries. Entries not yet present are then appended, which makes the operation idempotent if the LogRequest is duplicated. Finally, if *leaderCommit* exceeds the follower's *commitLength*, the newly committed entries are delivered to the application in log order.

Algorithm 6.7: Raft (7/9): updating followers' logs

```
1  function AppendEntries(prefixLen, leaderCommit, suffix)
2      if suffix.length > 0 ∧ log.length > prefixLen then
3          index := min(log.length, prefixLen + suffix.length) − 1
4          if log[index].term ≠ suffix[index − prefixLen].term then
5              log := ⟨log[0], log[1], . . . , log[prefixLen − 1]⟩
6      if prefixLen + suffix.length > log.length then
7          for i := log.length − prefixLen to suffix.length − 1 do
8              append suffix[i] to log
9      if leaderCommit > commitLength then
10         for i := commitLength to leaderCommit − 1 do
11             deliver log[i].msg to the application
12         commitLength := leaderCommit
```

The leader (algorithm 6.8) steps down to follower if a LogResponse carries a later term, meaning a new election has started, and ignores responses with outdated terms. On success it records the number of entries the follower acknowledged and tries to commit. On failure the follower's *logOk* was false, so the leader decrements *sentLength*[*follower*] and retries from one entry earlier. This may repeat, but eventually the leader sends entries that cleanly extend the follower's log and the follower accepts them (fewer retries are possible with a more complex algorithm).

Algorithm 6.8: Raft (8/9): leader receiving log acknowledgements

```
on receiving (LogResponse, follower, term, ack, success) at nodeId do
    if term = currentTerm ∧ currentRole = leader then
        if success = true ∧ ack ≥ ackedLength[follower] then
            sentLength[follower] := ack
            ackedLength[follower] := ack
            CommitLogEntries()
        else if sentLength[follower] > 0 then
            sentLength[follower] := sentLength[follower] − 1
            ReplicateLog(nodeId, follower)
    else if term > currentTerm then
        currentTerm := term
        currentRole := follower
        votedFor := null
        cancel election timer
```

Entries acknowledged by a quorum are ready to be committed by the leader (algorithm 6.9). The function *acks*(*length*) counts the nodes that have acknowledged at least *length* log entries. The set *ready* holds the prefix lengths acknowledged by a majority quorum; if its maximum exceeds *commitLength* (and the last entry of that prefix is from the current term), the leader delivers the new entries to the application and advances *commitLength*. The next LogRequest carries the new value, so the followers commit and deliver the same entries.

Algorithm 6.9: Raft (9/9): leader committing log entries

```
define acks(length) = |{n ∈ nodes | ackedLength[n] ≥ length}|
function CommitLogEntries
    minAcks := ⌈(|nodes| + 1)/2⌉
    ready := {len ∈ {1, ..., log.length} | acks(len) ≥ minAcks}
    if ready ≠ {} ∧ max(ready) > commitLength ∧ log[max(ready) − 1].term = currentTerm then
        for i := commitLength to max(ready) − 1 do
            deliver log[i].msg to the application
        commitLength := max(ready)
```

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 6 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 7: Replica Consistency

"Consistency" means different things in different contexts. In ACID transactions, a transaction moves the database from one consistent state to another, where "consistent" means satisfying application-specific invariants (e.g. "every course with students enrolled must have at least one lecturer"); this is a property of a state. In replication, it describes a relationship between replicas: should consistent replicas be in the same state (when exactly?), or should reads return the same result? Since there is no one true definition, we speak of consistency models, of which read-after-write consistency (Lecture 5) is one example.

7.1 Two-phase commit

A distributed transaction reads or writes data on multiple nodes, whether replicas of the same dataset or different parts of a larger one. Atomicity in ACID means a transaction either commits, making its updates durable, or aborts, leaving no visible side-effects; ACID consistency relies on it. For a distributed transaction this means all nodes must commit or all must abort, and if any node crashes, all must abort. Ensuring this is the atomic commitment problem. It looks a bit like consensus but differs in its details:

Consensus	Atomic commit
One or more nodes propose a value.	Every node votes whether to commit or abort.
Any one of the proposed values is decided.	Must commit if all nodes vote to commit; must abort if at least one node votes to abort.
Crashed nodes can be tolerated, as long as a quorum is working.	Must abort if a participating node crashes.

The most common atomic commitment algorithm is two-phase commit (2PC), not to be confused with two-phase locking (2PL), which ensures serializable isolation. (Three-phase commit assumes the unrealistic synchronous model.) A client starts a regular single-node transaction on each participating replica and performs the usual reads and writes. To commit, it sends a commit request to the transaction coordinator, a designated node that manages the protocol (sometimes part of the client) (figure 20). In the first phase, the coordinator sends a prepare message to each replica, and each replies whether it is able to commit. A replica does not commit yet, but it must make sure it will definitely be able to commit if instructed: before replying ok it writes all the transaction's updates to disk and checks integrity constraints, while still holding the transaction's locks. The coordinator decides to commit if all replicas reply ok, and to abort if any wants to abort or fails to reply within a timeout. In the second phase it sends the decision to every replica, which commits (guaranteed possible after the prepare) or rolls back.

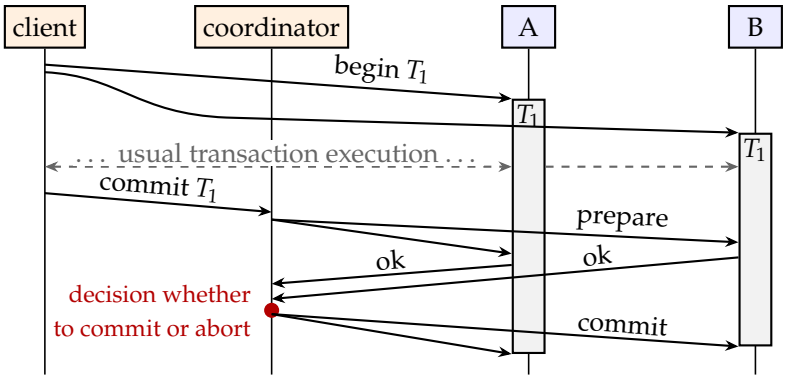


Figure 20: Two-phase commit of a transaction T_1 on replicas A and B.

The coordinator is a single point of failure. It writes its decision to disk and, on recovery, reads it and sends it to the replicas (or aborts if no decision was made before the crash). But if it crashes after the prepare and before broadcasting its decision, the other nodes do not know how it decided. Replicas that answered ok to the prepare can neither commit nor abort without risking violating atomicity, since that could contradict the coordinator and the other nodes; these IN-DOUBT transactions are blocked until the coordinator recovers.

The single point of failure can be avoided with consensus or total order broadcast. In a fault-tolerant 2PC based on PAXOS COMMIT (algorithm 7.1), every participating node disseminates its vote by total order broadcast. If a node suspects that a replica has failed (no vote within some timeout), it may vote to abort on that replica's behalf. This creates a race if the slow replica broadcasts its own vote to commit at about the same time, so each recipient counts only the first vote delivered from each replica and ignores later ones; total order broadcast guarantees all nodes agree on which vote that was. If the first delivered vote from some replica is to abort, the transaction aborts immediately; otherwise a node waits until it has delivered at least one vote from every replica, and commits if none of the first votes was to abort. All nodes thus make the same decision.

Algorithm 7.1: Fault-tolerant two-phase commit

```
1  on initialisation for transaction T do
2  |  commitVotes[T] := {}; replicas[T] := {}; decided[T] := false
3  on request to commit transaction T with participating nodes R do
4  |  for each r ∈ R do send (Prepare, T, R) to r
5  on receiving (Prepare, T, R) at node replicaId do
6  |  replicas[T] := R
7  |  ok := "is transaction T able to commit on this replica?"
8  |  total order broadcast (Vote, T, replicaId, ok) to replicas[T]
9  on a node suspects node replicaId to have crashed do
10 |  for each transaction T in which replicaId participated do
11 |  |  total order broadcast (Vote, T, replicaId, false) to replicas[T]
12 on delivering (Vote, T, replicaId, ok) by total order broadcast do
13 |  if replicaId ∉ commitVotes[T] ∧ replicaId ∈ replicas[T] ∧ ¬decided[T] then
14 |  |  if ok = true then
15 |  |  |  commitVotes[T] := commitVotes[T] ∪ {replicaId}
16 |  |  |  if commitVotes[T] = replicas[T] then
17 |  |  |  |  decided[T] := true
18 |  |  |  |  commit transaction T at this node
19 |  |  else
20 |  |  |  decided[T] := true
21 |  |  |  abort transaction T at this node
```

7.2 Linearizability

Atomic commitment makes all participants commit or abort, but when several nodes concurrently read and modify shared data, we also need to reason about the interaction of concurrent operations. Linearizability is the strongest option. Informally, every operation takes effect atomically sometime after it started and before it finished, and all operations behave as if executed on a single copy of the data, even if there are in fact multiple replicas. As a consequence every operation returns an "up-to-date" value, which is why linearizability is sometimes vaguely called strong consistency. It matters in shared-memory concurrency too: on a multi-core CPU each core has its own caches, so memory is not linearizable by default. Linearizability is not serializability. Serializability means transactions have the same effect as if executed in some serial order, without saying which order; linearizability defines which values operations must return, depending on their concurrency and relative ordering. A system with both provides strict serializability (or one-copy serializability).

Read-after-write consistency constrains reads after writes by the same node; linearizability generalises it to operations by different nodes. An operation starts when the application requests it and finishes when its result is returned (with quorums, when responses from a quorum have arrived). We draw the client's view of an operation as a rectangle from start to finish containing its effect: set(x, v) updates data item x to v, and get$(x) \to v$ is a read of x that returns v. Linearizability depends only on these start and finish times and outcomes, not on the implementation, and the key question is whether one operation finished before another started, regardless of the nodes involved.

If two gets start after a set(x, v_1) has finished, both must return v_1, even if the clients have never communicated. If a get overlaps in time with the set, either may take effect first, so the get may return v_1 or the previous value v_0.

"A finished before B started" is not "A happened before B": two operations that do not overlap in time can be concurrent under happens-before if no communication occurred between them. Linearizability is defined in terms of REAL TIME, as seen by a hypothetical global observer who sees all nodes instantaneously (or by perfectly synchronised clocks). No such observer exists with variable network latency, but defining linearizability this way means that a system proved linearizable keeps its guarantees whether or not communication has taken place.

Quorum reads and writes alone are not linearizable, because the order between two gets also matters (figure 21). Client 1 sets x to v_1; the update reaches A quickly but B and C late. Client 2 reads from the quorum $\{A, B\}$, receives v_1 and v_0, and returns the newer v_1. After client 2's read has finished, client 3 reads from the quorum $\{B, C\}$, receives v_0 from both, and returns v_0. Since client 2's operation finished before client 3's started, linearizability requires client 3 to observe a state no older than client 2 did, and v_0 is older than v_1.

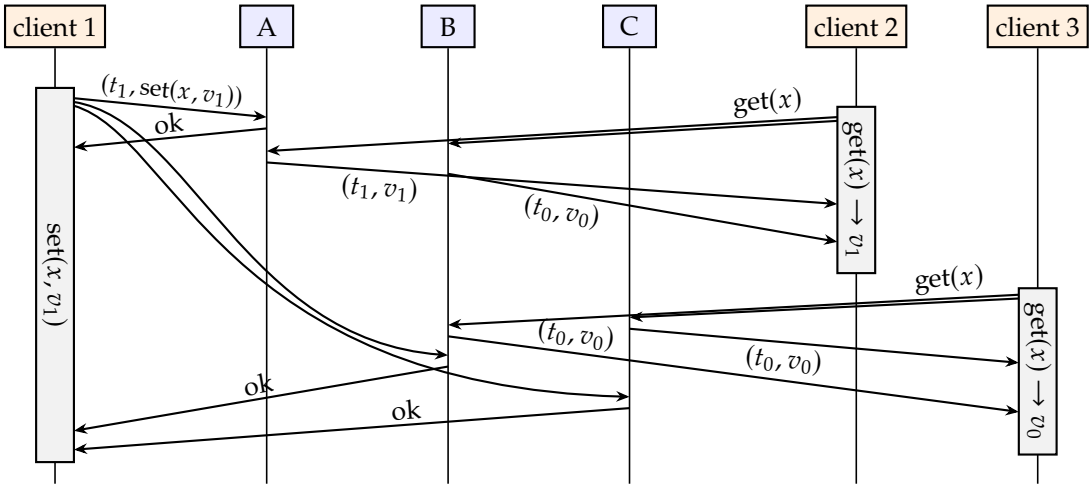


Figure 21: Not linearizable, despite quorum reads and writes: client 3's read starts after client 2's has finished, yet returns the older value v_0.

The ABD ALGORITHM (after Attiya, Bar-Noy, and Dolev) makes quorum reads and writes linearizable. Assume first that only one designated node performs set operations; a set sends the update to all replicas and waits for acknowledgements from a quorum, as before. A get sends the request to the replicas and waits for a quorum of responses. If some responses carry a more recent value than others (by timestamp), the client writes the most recent value back to all replicas that did not already respond with it, as in read repair, and the get finishes only once the most recent value is stored on a quorum, that is, once a quorum has either acknowledged the write-back or returned the most recent value in the first place (figure 22). When a get or set finishes, the value read or written is on a quorum, so every subsequent quorum operation sees that value or a later one.

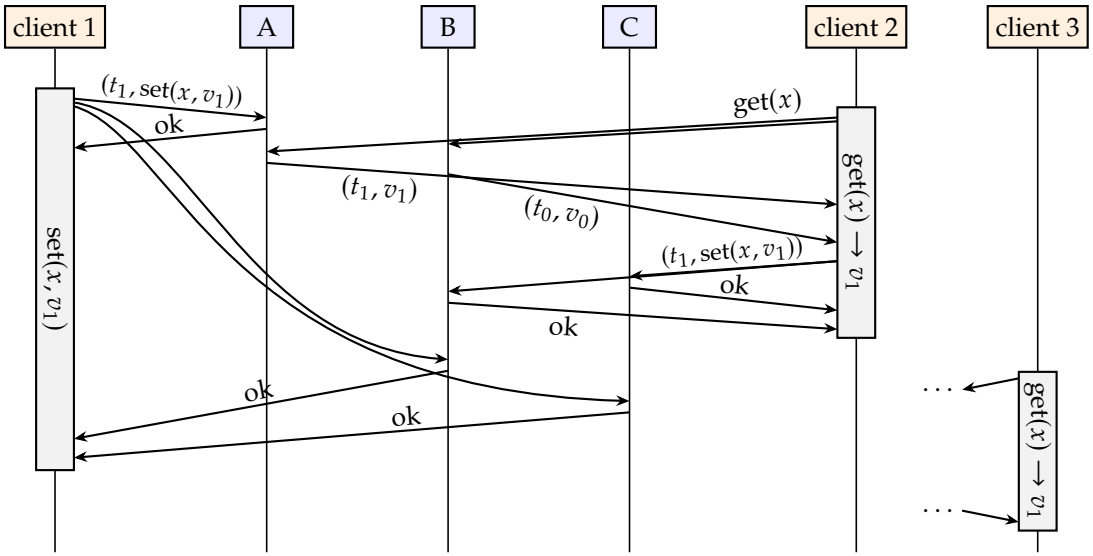


Figure 22: ABD: before returning, client 2 writes (t_1, v_1) back to B and C and waits for a quorum to hold it; client 3's later read then returns v_1.

With several writers, timestamps must reflect the real-time order: if set(x, v_1) with timestamp t_1 finishes before set(x, v_2) with timestamp t_2 starts, then $t_1 < t_2$. So each set first requests the latest timestamp from the replicas and waits for a quorum (like a get), and uses one plus the maximum timestamp received; the quorum contains at least one replica that saw any completed set. Concurrent sets may then get the same timestamp, so each client has a unique ID that is part of its timestamps, and a get that sees equal timestamps with different client IDs picks the winner by a total order on client IDs (as with Lamport timestamps). This gives linearizable get and set for any number of nodes.

These sets are BLIND WRITES (unconditional writes): concurrent writes to the same item are resolved by last writer wins, and the losing values are silently discarded. To overwrite a value only if it has not been modified concurrently, we want an atomic COMPARE-AND-SWAP: CAS$(x, \textit{oldValue}, \textit{newValue})$ sets x to *newValue* if and only if the current value of x is *oldValue*. ABD cannot implement CAS, because different replicas may see operations in different orders and disagree about whether a CAS succeeded. Total order broadcast can (algorithm 7.2): broadcast every operation and execute it when it is delivered, so, as in state machine replication, it has the same effect and outcome on every replica.

Algorithm 7.2: Linearizable compare-and-swap

```
1  on request to perform get(x) do
2  |  total order broadcast (get, x) and wait for delivery
3  on request to perform CAS(x, old, new) do
4  |  total order broadcast (CAS, x, old, new) and wait for delivery
5  on delivering (get, x) by total order broadcast do
6  |  return localState[x] as result of operation get(x)
7  on delivering (CAS, x, old, new) by total order broadcast do
8  |  success := false
9  |  if localState[x] = old then
10 |  |  localState[x] := new; success := true
11 |  return success as result of operation CAS(x, old, new)
```

7.3 Eventual consistency

Linearizability makes a distributed system behave as if it were not distributed, which is simple for applications, but it has costs. Performance: ABD and the total-order CAS send many messages and wait for responses. Scalability: when all updates are sequenced through a leader, as in Raft, the leader can become a bottleneck. Availability, perhaps the biggest problem: every operation needs a quorum, so a node that cannot contact enough replicas cannot process any operation and is effectively unavailable, even though it is running. A calendar app that syncs across a phone, tablet, and computer should let you view, add, and modify events while a device is offline (say, with poor mobile coverage), and sync the updates in the background later; with linearizable replication this would be impossible, since an offline device cannot reach a quorum.

Theorem 7.1 [CAP theorem**].** *If there is a network partition in a system, it must choose between linearizable consistency, in which case some replicas cannot respond to requests because they cannot communicate with a quorum, and availability, in which case replicas respond even without communicating with other replicas, and linearizability cannot be guaranteed.*

The name stands for consistency, availability, and partition tolerance. The framing "pick 2 out of 3" is misleading: a system can be both linearizable and available as long as there is no partition, and the choice is forced only during a partition. In figure 23, C cannot communicate with A and B. On their side of the partition, A and B form a quorum, so linearizable operations continue; C must either wait, potentially indefinitely, for the partition to be repaired, or return its local value v_0, which does not reflect the value v_1 written by A.

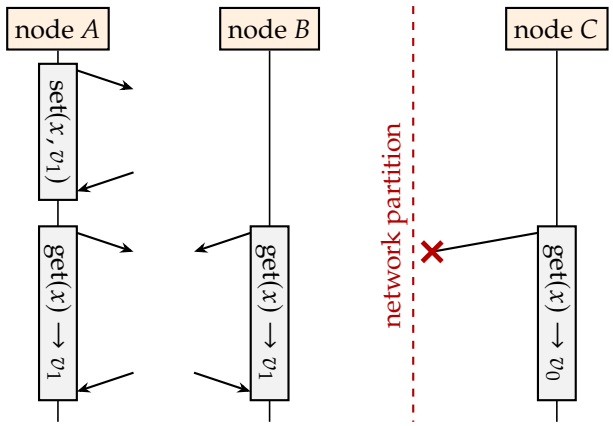


Figure 23: The CAP trade-off: during a partition, A and B serve linearizable operations, while C must wait or return the stale value v_0.

The calendar app chooses availability, as do many other systems. Letting each replica process reads and writes based only on its local state, without waiting for communication with other replicas, is called OPTIMISTIC REPLICATION.

Definition: EVENTUAL CONSISTENCY: "if no new updates are made to an object, eventually all reads will return the last updated value." STRONG EVENTUAL CONSISTENCY requires

- *eventual delivery:* every update made to one non-faulty replica is eventually processed by every non-faulty replica;
- *convergence:* any two replicas that have processed the same set of updates are in the same state, even if they processed the updates in different orders.

Eventual consistency is very weak: if updates never stop, its premise is never true, and it gives no bound on how long convergence takes. Strong eventual consistency is often more appropriate: as replicas communicate, they converge towards the same state. It does not require waiting for network communication, and causal broadcast (or weaker) can disseminate the updates. Concurrent updates to the same object cause CONFLICTS, which must be resolved. For example, if the time of a calendar event is changed on one device while its title is concurrently changed on another, last writer wins converges by keeping one update and discarding the other (a small data loss); a more refined approach merges both changes.

The problems seen so far differ in the minimum system-model assumptions they need, summarised in the table below from the strongest assumptions to the weakest. Atomic commit must wait for all nodes participating in a transaction (potentially all nodes in the system). Consensus, total order broadcast, and linearizable operations need only a quorum and so tolerate some unavailable nodes. By the FLP result, consensus and total order broadcast require partial synchrony, and since linearizable CAS is equivalent to consensus, so does it. ABD, in contrast, is asynchronous, using no clocks or timeouts. Eventual and strong eventual consistency process operations without waiting for other nodes and without timing assumptions; likewise, in causal and weaker broadcasts a node

delivers its own message immediately, just as a replica processes its own operations immediately. Like complexity classes of algorithms (sorting generally takes $O(n \log n)$), this hierarchy captures the unavoidable minimum communication and synchrony requirements of common problems.

Problem	Must wait for communication	Requires synchrony
atomic commit	all participating nodes	partially synchronous
consensus, total order broadcast, linearizable CAS	quorum	partially synchronous
linearizable get/set	quorum	asynchronous
eventual consistency, causal broadcast, FIFO broadcast	local replica only	asynchronous

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 7 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.

Lecture 8: Case Studies

Two kinds of practical systems that manage concurrent access to data build on the concepts above: collaboration software, which favours availability, and Google's Spanner, which provides the strongest consistency.

8.1 Collaboration and conflict resolution

Collaboration software lets several people work on a shared file or document: Google Docs and Office 365 (text documents, spreadsheets, presentations), Overleaf (collaborative LaTeX), multi-user graphics software such as Figma, project planning tools such as Trello, note-taking apps such as OneNote, Evernote, and Notion, and shared calendars. Each user's device holds a local replica of the data, which the user can update at any time, even offline, and which syncs with the others when the network is available; the challenge is to reconcile concurrent updates. Linearizable replication would be slow, since every operation would contact a quorum, and would not work offline, so most collaboration software uses optimistic replication with strong eventual consistency. Two families of algorithms are used: conflict-free replicated data types (CRDTs), which are operation-based or state-based, and operational transformation (OT).

Example: *Calendar sync.* Two nodes start with the same calendar entry {title: "Lecture", date: "2020-11-05", time: "12:00"}. During a network partition, node A changes the title to "Lecture 1" and node B changes the time to "10:00". When the nodes sync, both end with {title: "Lecture 1", date: "2020-11-05", time: "10:00"}, reflecting both changes.

This is conflict resolution: several concurrent writes to the same object are integrated into a single final state. A CRDT is a replicated object that an application accesses through the interface of an abstract datatype, such as a set, list, map, tree, graph, or counter. In a map CRDT (algorithm 8.1), the application reads the value for a key, or sets the value for a key (adding the key if absent). Each node's state is a set *values* of (*timestamp*, *key*, *value*) triples, with at most one triple per key, so a read, which is purely local, is unique if it exists. To set a key, a node creates a globally unique timestamp (a Lamport timestamp is a good choice) and broadcasts the timestamp, key, and value. On delivery, the update is ignored if *values* already holds the key with a higher timestamp; otherwise the previous triple for the key (if any) is replaced. Concurrent updates to the same key are thus resolved by last writer wins.

Algorithm 8.1: Operation-based map CRDT

```
1  on initialisation do
2  |  values := {}
3  on request to read value for key k do
4  |  if ∃t, v. (t, k, v) ∈ values then return v else return null
5  on request to set key k to value v do
6  |  t := newTimestamp()                    // globally unique, e.g. Lamport timestamp
7  |  broadcast (set, t, k, v) by reliable broadcast (including to self)
8  on delivering (set, t, k, v) by reliable broadcast do
9  |  previous := {(t', k', v') ∈ values | k' = k}
10 |  if previous = {} ∨ ∀(t', k', v') ∈ previous. t' < t then
11 |  |  values := (values \ previous) ∪ {(t, k, v)}
```

Reliable broadcast may deliver the updates

$$(\mathsf{set}, t_1, \text{“title”}, \text{“Lecture 1”}) \quad \text{and} \quad (\mathsf{set}, t_2, \text{“time”}, \text{“10:00”})$$

in either order. The algorithm still provides strong eventual consistency: reliable broadcast delivers every operation to every non-crashed replica (eventual delivery), and applying an operation is commutative, so the order of delivery does not matter (convergence). This realises replication over reliable broadcast without total order. It is an OPERATION-BASED CRDT, since each message describes an update operation, and operations complete without network connectivity, because the sender of a reliable broadcast delivers to itself immediately and sends to the others later.

A STATE-BASED CRDT instead broadcasts the whole replica state and merges it into the recipient's state (algorithm 8.2). For the map, reading is as before; a set updates *values* directly and broadcasts all of *values*, and the merge operator keeps, for each key, the triple with the greatest timestamp:

$$s_1 \sqcup s_2 = \{(t, k, v) \in (s_1 \cup s_2) \mid \nexists (t', k', v') \in (s_1 \cup s_2).\ k' = k \wedge t' > t\}.$$

Algorithm 8.2: State-based map CRDT

```
1  on initialisation do
2  |  values := {}
3  on request to read value for key k do
4  |  if ∃t, v. (t, k, v) ∈ values then return v else return null
5  on request to set key k to value v do
6  |  t := newTimestamp()                    // globally unique, e.g. Lamport timestamp
7  |  values := {(t', k', v') ∈ values | k' ≠ k} ∪ {(t, k, v)}
8  |  broadcast values by best-effort broadcast
9  on delivering V by best-effort broadcast do
10 |  values := values ⊔ V
```

The merge operator $\sqcup$ must be, for all states s_1, s_2, s_3,

- commutative: $s_1 \sqcup s_2 = s_2 \sqcup s_1$;

- associative: $(s_1 \sqcup s_2) \sqcup s_3 = s_1 \sqcup (s_2 \sqcup s_3)$;
- idempotent: $s_1 \sqcup s_1 = s_1$.

Operation-based CRDTs typically have smaller messages. State-based CRDTs tolerate message loss and duplication: as long as two replicas eventually exchange their latest states they converge, even if earlier messages were lost, and duplicates are harmless because merging is idempotent. So a state-based CRDT can use unreliable best-effort broadcast, while an operation-based CRDT needs reliable broadcast (some even need causal broadcast). State-based CRDTs need not use broadcast at all: they can also merge concurrent updates in quorum replication, anti-entropy, and similar schemes.

In a collaborative editor such as Google Docs, keystrokes apply immediately to the local copy of the document in the browser, without waiting for a server or other users, so the documents of users typing concurrently temporarily diverge and must converge as they communicate. Treat a plain-text document as a list of characters in which a user can insert or delete a character at any index (formatting, images, and tables add complexity). In figure 24(a), both users start with "BC". User A inserts "A" at index 0, giving "ABC", while user B concurrently inserts "D" at index 2, giving "BCD"; we expect both to end with "ABCD". B applies A's (insert, 0, "A") and gets "ABCD", but A applies B's (insert, 2, "D") and gets "ABDC": when B inserted at index 2 it meant the position after "C", but A's concurrent insertion at index 0 shifted every later character by one, so that position is now index 3.

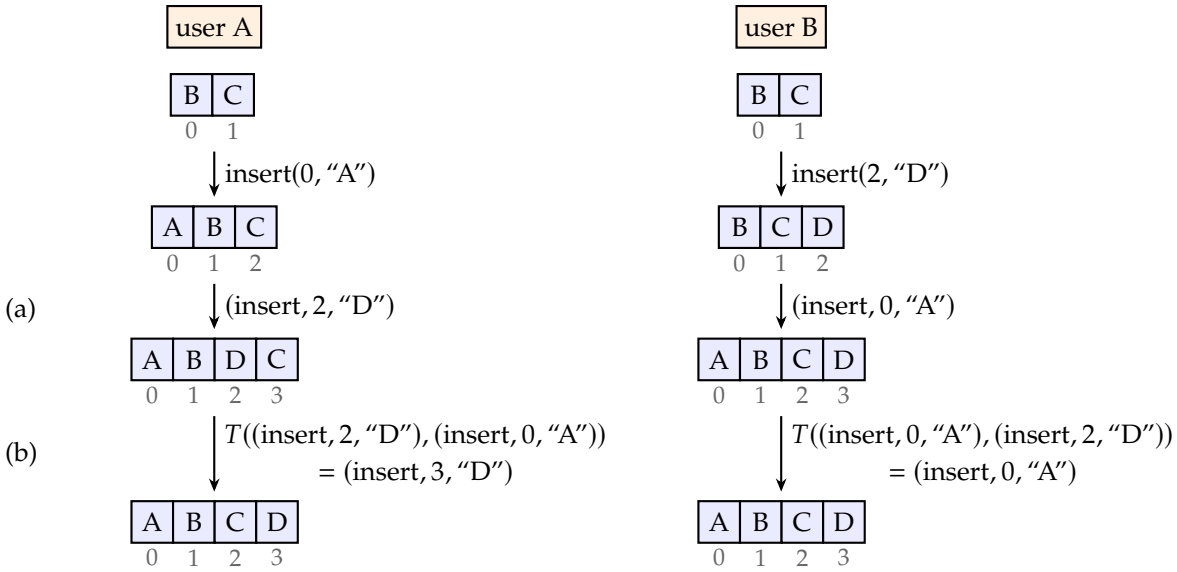


Figure 24: Concurrent insertions into "BC", with indexes shown below the characters. (a) Applying the other user's operation unchanged makes the replicas diverge. (b) Operational transformation makes both reach "ABCD".

Operational transformation is a family of algorithms that differ in how they resolve conflicts but share a principle (figure 24b). A node keeps the history of operations it has performed.

When it receives an operation concurrent with one or more of its own, it transforms the incoming operation relative to them: $T(op_1, op_2)$, for an incoming op_1 and a concurrent local op_2, returns an operation op_1' whose effect on the local state is the one op_1 intended. Here $T((\text{insert}, 2, \text{“D”}), (\text{insert}, 0, \text{“A”})) = (\text{insert}, 3, \text{“D”})$, because the insertion at index 0 moved the target position, while $T((\text{insert}, 0, \text{“A”}), (\text{insert}, 2, \text{“D”})) = (\text{insert}, 0, \text{“A”})$ is unchanged, since an insertion later in the document does not affect index 0. Deletions and formatting make the transformation more complicated. The conflict resolution of Google Docs uses OT, based on the Jupiter system from Xerox PARC. A limitation is that OT requires the users' communication to use total order broadcast, through a designated leader that sequences updates or a consensus algorithm.

A CRDT for text editing avoids total order broadcast. Instead of identifying positions by index, which necessitates transformation, it gives each character a unique identifier that never changes as characters are inserted or deleted around it. In one construction, each character gets a rational number $i \in \mathbb{Q}$ with $0 < i < 1$, where 0 is the beginning of the document and 1 the end, and characters appear in ascending order of their numbers. The symbols $\vdash$ and $\dashv$ mark the beginning and the end; they are part of the algorithm's internal state, not visible to the user. To insert between adjacent characters with positions i and j, the new character gets position $\frac{i+j}{2}$, which always lies between them, provided we use arbitrary-precision arithmetic (floating-point numbers run out of precision once the intervals get small). Two nodes inserting concurrently at the same place may generate the same position, so ties are broken by the ID of the node that generated the character. An insertion is simply broadcast, and replicas add the character to their set and sort by position (figure 25).

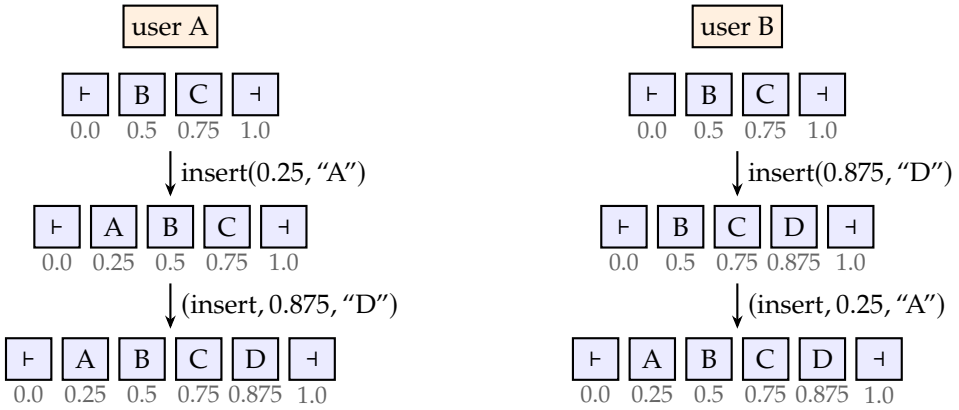


Figure 25: Text CRDT: each character carries a position number (below it), so concurrent insertions commute and both replicas converge.

In algorithm 8.3, a replica's state is the set *chars* of (*position*, *nodeId*, *character*) triples, initialised with the triples for $\vdash$ and $\dashv$. ElementAt iterates over *chars* in ascending order of position: it finds the minimum element (no other element has a lower position, ties broken by the lowest *nodeId*), returns it if *index* = 0, and otherwise removes it, decrements the index, and repeats (a rather slow algorithm; a real implementation would be more efficient). Reading the character at an index adds 1 to skip $(0, \text{null}, \vdash)$. An insertion takes the positions p_1 and p_2 of the immediate predecessor and successor

and broadcasts the new position $(p_1 + p_2)/2$ by causal broadcast; delivery adds the triple to *chars*. A deletion broadcasts the position and node ID that uniquely identify the character, and delivery removes the matching element, if it exists.

Algorithm 8.3: Operation-based text CRDT

```
1  function ElementAt(chars, index)
2  |   min := the unique triple (p, n, v) ∈ chars such that
   |     ∄(p', n', v') ∈ chars. p' < p ∨ (p' = p ∧ n' < n)
3  |   if index = 0 then return min else return ElementAt(chars \ {min}, index − 1)
4  on initialisation do
5  |   chars := {(0, null, ⊢), (1, null, ⊣)}
6  on request to read character at index index do
7  |   let (p, n, v) := ElementAt(chars, index + 1); return v
8  on request to insert character v at index index at node nodeId do
9  |   let (p1, n1, v1) := ElementAt(chars, index)
10 |   let (p2, n2, v2) := ElementAt(chars, index + 1)
11 |   broadcast (insert, (p1 + p2)/2, nodeId, v) by causal broadcast
12 on delivering (insert, p, n, v) by causal broadcast do
13 |   chars := chars ∪ {(p, n, v)}
14 on request to delete character at index index do
15 |   let (p, n, v) := ElementAt(chars, index + 1)
16 |   broadcast (delete, p, n) by causal broadcast
17 on delivering (delete, p, n) by causal broadcast do
18 |   chars := {(p', n', v') ∈ chars | ¬(p' = p ∧ n' = n)}
```

Causal broadcast (rather than merely reliable broadcast) ensures that every replica processes the insertion of a character before its deletion, which is necessary because inserting and deleting the same character do not commute. Insertions and deletions of different characters commute, so the algorithm converges and provides strong eventual consistency.

8.2 Google's Spanner

Despite its name, strong eventual consistency is fairly weak: a read need not return the most up-to-date value, since updates take time to propagate. Google's Spanner database provides the strongest consistency guarantees, at a cost that is right for some applications: serializable transaction isolation, linearizable reads and writes, and atomic commit of transactions across shards, each holding a subset of the data. It remains very scalable, with millions of nodes, petabytes of data, large transaction throughput, and replicas in datacenters worldwide. Many of its techniques are standard: state machine replication with Paxos within a shard, two-phase locking for serializability, and two-phase commit for cross-shard atomicity. The unusual part is its use of atomic clocks, which enables read-only transactions that need no locks.

Some read-only transactions read a large part of the database, such as a backup or an audit. With two-phase locking, a read lock on the whole database would block all writes for the duration, so it is important that large read-only transactions run in the background without locks and without

interfering with concurrent read-write transactions. Spanner lets a read-only transaction observe a CONSISTENT SNAPSHOT: the database as it was at a single point in time, even if it runs for a long time while parts of the database are updated. "Consistent" means consistent with causality: if $T_1 \rightarrow T_2$ (e.g. T_2 reads data written by T_1), a snapshot that reflects the writes of T_2 also reflects those of T_1, and a snapshot that does not reflect T_1's writes does not reflect T_2's either.

Snapshots are implemented with MULTI-VERSION CONCURRENCY CONTROL (MVCC), a form of optimistic concurrency control. Each read-write transaction T_w has a commit timestamp t_w, and every value is tagged with the timestamp of the transaction that wrote it; an update does not overwrite the old value but keeps old versions alongside the latest. A read-only transaction T_r has a snapshot timestamp t_r; it ignores values with $t_w > t_r$ and observes the most recent value with $t_w \leq t_r$. For snapshots to be consistent with causality, $T_1 \rightarrow T_2$ must imply $t_1 < t_2$; what makes Spanner special is how it assigns timestamps.

Physical timestamps may be inconsistent with causality. Lamport clocks are no better here, because linearizability depends on real-time order. If a user observes the results of T_1, executed on replica A, and then takes an action executed as T_2 on replica B, then T_1 finished before T_2 started, so T_2 should get a greater timestamp. But no message need pass from A to B: the communication goes via the user, who cannot be expected to attach a properly formed timestamp to every action, and without a reliable way of propagating timestamps on every communication step, logical timestamps cannot provide the ordering. A single server assigning timestamps would be a single point of failure and a bottleneck in a globally distributed database, and transactions on other continents would wait for intercontinental round trips.

TRUETIME is a system of physical clocks that returns not a single timestamp but an uncertainty interval $[t_{\text{earliest}}, t_{\text{latest}}]$. Perfect synchronisation is impossible, but the errors introduced along the way can be tracked: manufacturers report the error bounds of atomic clocks, the error of a GPS receiver depends on the quality of the satellite signals in range, the error of synchronisation over a network depends on the round-trip time, and the error of a quartz clock depends on its drift rate and the time since its last sync. Taking all of these into account, TrueTime ensures that the true physical time t_{real} satisfies $t_{\text{earliest}} \leq t_{\text{real}} \leq t_{\text{latest}}$ with very high probability (figure 26).

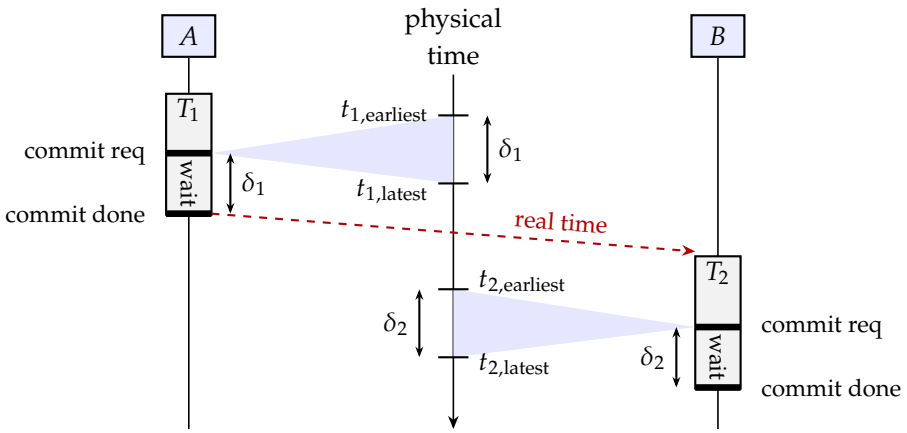


Figure 26: Commit wait in TrueTime: each transaction waits out its uncertainty δ_i before committing, so the intervals of T_1 and T_2 do not overlap.

When transaction T_i wants to commit, it gets an interval $[t_{i,\text{earliest}}, t_{i,\text{latest}}]$ from TrueTime and takes $t_{i,\text{latest}}$ as its commit timestamp. Before actually committing and releasing its locks, it waits for the uncertainty

$$\delta_i = t_{i,\text{latest}} - t_{i,\text{earliest}},$$

and only then do its writes become visible. This ensures that the commit timestamp is less than the true physical time when the transaction commits. So if T_2 starts later in real time than T_1 committed, even the earliest possible timestamp of T_2 exceeds T_1's timestamp: the waiting keeps the intervals of T_1 and T_2 from overlapping, even on different nodes with no communication between the transactions.

Every transaction waits for its uncertainty interval, so it must be kept small. Google installs atomic clocks and GPS receivers in every datacenter; the clock servers report their own uncertainty, and every node syncs its quartz clock with a server in the local datacenter every 30 seconds, where round trips are usually below 1 ms. If network latency increases (e.g. through congestion), the uncertainty interval grows accordingly. Between syncs a node's clock depends only on its quartz oscillator, and to be safe Google assumes a worst-case drift of 200 ppm, much more than is observed under normal conditions, and monitors each node's drift, alerting administrators to outliers. (According to the Spanner paper, bad CPUs are 6 times more likely than bad clocks, so TrueTime is as trustworthy as any other software Spanner depends on.) Thirty seconds after a sync, 200 ppm of drift amounts to $200 \cdot 10^{-6} \times 30\,\text{s} = 6\,\text{ms}$, on top of the uncertainty from network latency, GPS receivers, and atomic clocks. The uncertainty therefore grows gradually to about 7 ms and resets to about 1 ms (round-trip time plus clock server uncertainty) at every sync (figure 27).

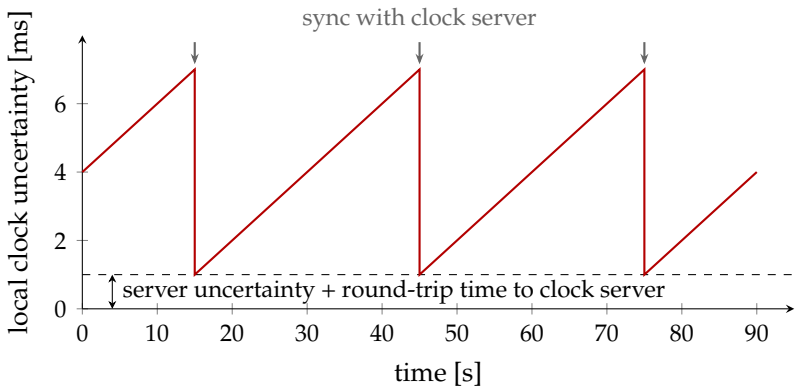


Figure 27: TrueTime's local clock uncertainty between syncs with the clock server, assuming a worst-case drift of 200 ppm.

The average uncertainty, and hence the average commit wait, is about 4 ms under normal conditions, much faster than waiting for an intercontinental round trip (on the order of 100 ms or more). In summary, TrueTime gives upper and lower bounds on the current physical time through careful accounting of uncertainty, keeps the interval small with high-precision clocks, and by waiting out the uncertainty Spanner gets timestamps consistent with causality; using them for MVCC gives serializable transactions with lock-free read-only transactions, fast, and without requiring clients to propagate logical timestamps.

Source. Martin Kleppmann, *Distributed Systems* (Cambridge Part IB), Lecture 8 notes (Michaelmas 2021). Adapted under a Creative Commons BY-SA license.